10X Front-end სადიპლომო პროექტი - „10X Shop“

Product Requirements Document (PRD)

ინდივიდუალური სადიპლომო პროექტი

ვერსია 1.1 · 2026 წლის ნაკადი

დანართი

სარჩევი

დანართი A – სრული ტექნიკური სპეციფიკაცია

A1. პროდუქტის მიმოხილვა

A1.1 მომხმარებელი და პრობლემა
A1.2 გვერდების რუკა (Route Map)
A1.3 ძირითადი ფლოუ (Happy Path)
A1.4 არქიტექტურა და მონაცემთა ნაკადი
A1.5 მონაცემთა მოდელები და localStorage რეესტრი
A1.6 მონაცემების წყარო - DummyJSON API
A1.7 ტექნიკური შეზღუდვები

A2. გვერდების დეტალური სპეციფიკაციები

P0 - გლობალური წესები (ყველა გვერდზე) [CORE]
P1 - Login გვერდი (/login) [CORE]
P2 - Catalog გვერდი (/) [ბირთვი CORE; კატეგორია / სორტი / URL-state FULL]
P3 - Product გვერდი (/products/:id) [ბირთვი CORE; reviews / related FULL]
P4 - Cart გვერდი (/cart) [CORE; promo კოდი FULL]
P5 - Checkout გვერდი (/checkout) [ბირთვი CORE; პირობითი ბარათის ველები FULL]
P6 - Orders გვერდი (/orders, /orders/:id) [FULL]
P7 - Profile გვერდი (/profile) [FULL]

A3. მოდულური მოთხოვნები: TypeScript (T), ოპტიმიზაცია (O), პროცესი (D)

T - TypeScript [მოდული: დინამიური ვებგვერდის შექმნა TypeScript-ის საშუალებით]
O - ოპტიმიზაცია, Debugging და თავსებადობა [მოდული: ვებ გვერდის ოპტიმიზაცია]
D - დაგეგმვა, პროცესი და ანგარიშგება [მოდული: პრაქტიკული პროექტი]

A3-TW. Tailwind CSS — ახალი მოდული, CORE/FULL პრიორიტეტები და სამკვირიანი განრიგი (A3-ის შემდეგ)

დანართი A – სრული ტექნიკური სპეციფიკაცია

ეს ნაწილი საცნობაროა: ყოველი ნაბიჯი აქ არის დეტალურად აღწერილი – ზუსტი ველები, ტექსტები, ვალიდაციის წესები, API-ს მისამართები, ტიპები და თითო გვერდის Measurable Outcome. მუშაობისას სწორედ აქ იხედები;

დანართის კლასიფიკაცია: CORE = მინიმალური ჩათვლა; FULL = გაფართოებული ფუნქციონალი.

A1. პროდუქტის მიმოხილვა

A1.1 მომხმარებელი და პრობლემა

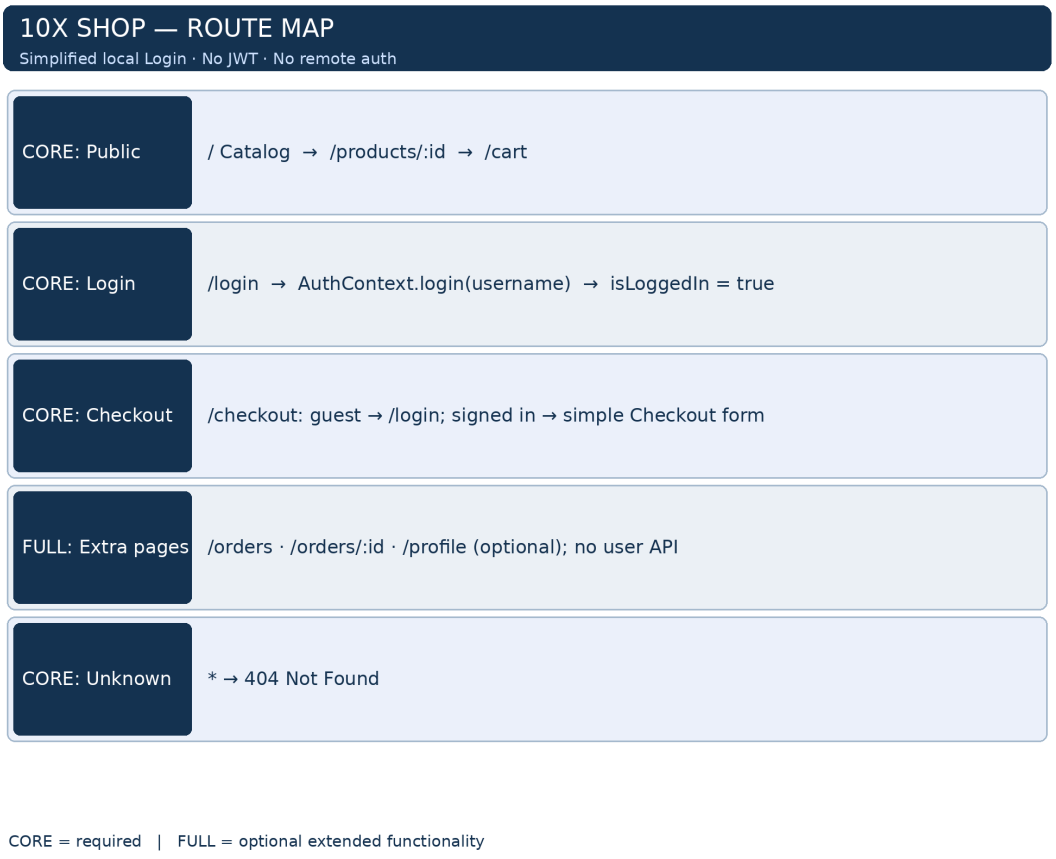
ჩვენი მომხმარებელია **მყიდველი** – ადამიანი, რომელსაც უნდა სწრაფად იპოვოს პროდუქტი, შეადაროს ფასები და ფასდაკლებები, შეაგროვოს კალათაში, გააფორმოს შეკვეთა და ნახოს, სად არის მისი შეკვეთა. მაღაზიისთვის კი მნიშვნელოვანია, რომ კატალოგი სწრაფად იტვირთებოდეს (ნელი გვერდი = დაკარგული გაყიდვა), ყველა მოწყობილობასა და ბრაუზერში მუშაობდეს და შეცდომებზე „არ ტყდებოდეს“. 10X Shop აძლევს მყიდველს: კატალოგს ძებნით, კატეგორიებითა და სორტით; პროდუქტის დეტალურ გვერდს შეფასებებით; კალათას promo კოდით; შეკვეთის გაფორმებას ვალიდაციით; შეკვეთების ისტორიას სტატუსებით და პროფილს.

A1.2 გვერდების რუკა (Route Map)

პროდუქტი ერთი HTML გვერდია (SPA) – ყველა „გვერდი“ react-router-ის მარშრუტია, ერთ საერთო Layout-ში (Header + Footer):

მარშრუტი	გვერდი	წვდომა	დონე
/login	P1 – Login	საჯარო	CORE
/	P2 – Catalog	საჯარო	CORE + FULL
/products/:id	P3 – Product	საჯარო	CORE + FULL
/cart	P4 – Cart	საჯარო	CORE
/checkout	P5 – Checkout	დაცული	CORE
/orders, /orders/:id	P6 – Orders	დაცული	FULL
/profile	P7 – Profile	დაცული	FULL
*	404 – Not Found	საჯარო	CORE

კატალოგი, პროდუქტის დეტალები და კალათა საჯაროა. Checkout-ზე გადასასვლელად საკმარისია აპლიკაციის ფარგლებში მარტივი Login (React state/Context); თუ მომხმარებელი შესული არ არის, გადადის /login-ზე, ხოლო შესვლის შემდეგ — /checkout-ზე.



CORE Happy Path: Catalog → Product Details → Cart (add/quantity/remove/total) → მარტივი Login (საჭიროებისას) → Checkout controlled form → შეკვეთის დადასტურება. FULL: Search/Filtering/Sorting/Pagination, Orders, Profile, Reviews, Related products, Promo და თემა.

A1.3 ძირითადი ფლოუ (Happy Path)

CORE ფლოუ აღწერილია ზემოთ; ქვემოთ მოცემული ექვსნაბიჯიანი სცენარი აჩვენებს FULL ფუნქციების დამატებით შესაძლებლობებს.

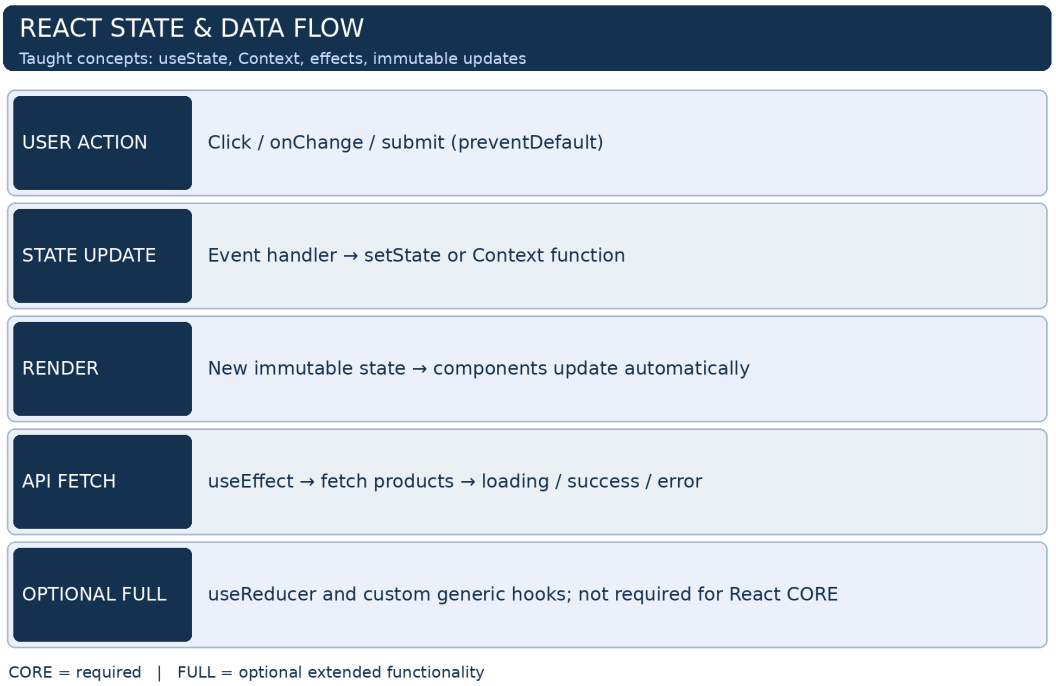
- 1. მომხმარებელი ხსნის საიტს -> ხედავს კატალოგს (12 პროდუქტი, პაგინაცია) -> ეძებს „phone“-ს, ირჩევს კატეგორიას, სორტავს ფასით.
- 2. აჭერს პროდუქტს -> ხედავს გალერეას, ფასდაკლებულ ფასს, მარაგს და შეფასებებს -> ამატებს კალათაში 2 ცალს.
- 3. კალათაში ცვლის რაოდენობას და გადადის Checkout-ზე; საჭიროებისას ხსნის /login-ს.
- 4. ავსებს მარტივ Login ფორმას, რის შემდეგაც აპლიკაცია იმახსოვრებს მომხმარებლის სახელს React Context-ში; Checkout-ში controlled form-ის ვალიდაციის შემდეგ ჩანს „Thank you!“ დადასტურება და კალათა ცარიელდება.
- 5. შეკვეთების გვერდზე ხედავს შეკვეთას სტატუსით Processing -> რამდენიმე წუთში Shipped -> Delivered.
- 6. პროფილზე ასწორებს სახელს, ცვლის თემას -> გამოდის (Logout) -> ისევ ლოგინდება -> შეკვეთები და კალათა ადგილზეა.

A1.4 არქიტექტურა და მონაცემთა ნაკადი

CORE არქიტექტურა: კომპონენტები, ტიპიზებული props/state/API მონაცემები, JSX, useState/useEffect, Router, Context, Cart-ის immutable updates, API-დან პროდუქტის წამოღება და Login/Checkout controlled forms. FULL: დამატებითი custom hooks და უფრო კომპლექსური კომპონენტური სტრუქტურა.

ოქროს ციკლი 2.0 - დაიმახსოვრე ეს ერთი სურათი

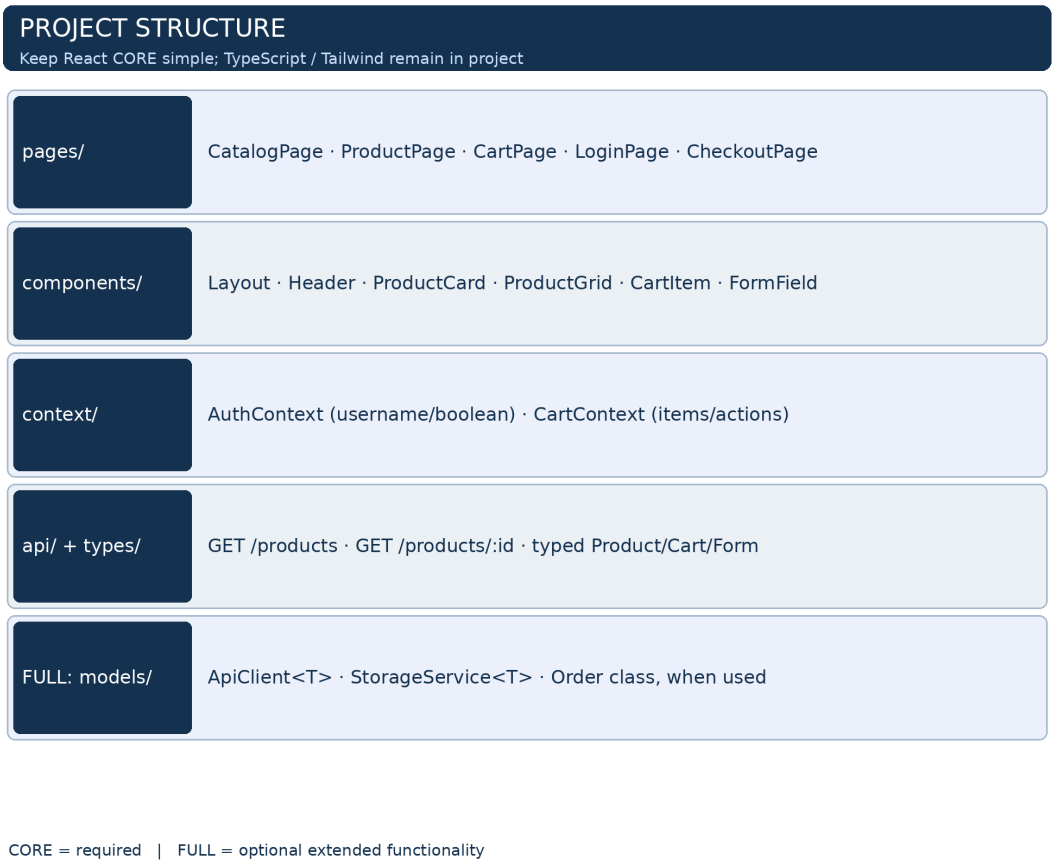
React-ის საბაზისო მონაცემთა ნაკადი: მომხმარებლის მოქმედება → event handler → setState/Context-ის ფუნქცია → ახალი immutable state → React-ის ხელახალი render. API მოთხოვნა იშვება useEffect-ში; loading/error/empty მდგომარეობები conditional rendering-ით გამოისახება. აუცილებლობის შემთხვევაში effect ასრულებს cleanup-ს.



დიაგრამა 3. React CORE: event handlers → useState/Context → immutable updates → rendering;

ფენები და ფოლდერები. პროექტი ფენებად იყოფა – თითო ფოლდერს ერთი პასუხისმგებლობა აქვს. ეს სტრუქტურა რეკომენდებულია (სახელები შეგიძლია შეცვალო, პრინციპი – არა):

```
src/
 api/       products.api.ts (API ფუნქციები); ApiClient.ts — დამატებითი არქიტექტურული მაგალითი
 types/     product.ts, user.ts, cart.ts, order.ts, api.ts
 models/    Cart.ts, Order.ts, StorageService.ts     // მხოლოდ FULL კლასების მაგალითები
 context/   AuthContext.tsx, CartContext.tsx; ThemeContext.tsx — FULL
 hooks/     დამატებითი useFetch.ts, useDebounce.ts და useLocalStorage.ts — FULL
 // CORE მონაცემების ჩამოტვირთვა შეიძლება პირდაპირ useEffect + fetch-ით
 components/  ui/ layout/ product/ cart/    // ხელახლა გამოყენებადი
 pages/     CatalogPage.tsx, ProductPage.tsx, CartPage.tsx, ...
 routes/    AppRoutes.tsx; ProtectedRoute.tsx — მარტივი სასწავლო redirect (CORE)
 utils/     format.ts, validation.ts     // მარტივი ვალიდაციის ფუნქციები
```



დიაგრამა 4. CORE კომპონენტები/კონტექსტი, API და ტიპები; დამატებითი OOP ფენები განეკუთვნება FULL-ს.

არქიტექტურის წესები (დაცვაზე მოწმდება):

- pages/ აერთიანებს reusable კომპონენტებს, props-ს და შესაბამის state-ს; კოდი ლოგიკურადაა დაყოფილი. რთული ფენობრივი აბსტრაქცია CORE-ში სავალდებულო არ არის.
- API-ის პროდუქტის მონაცემები ჩაიტვირთოს fetch-ით/useEffect-ით ან ცალკე API ფუნქციით; უნივერსალური ApiClient მხოლოდ გაფართოებული ვარიანტის შესაძლო გადაწყვეტაა.
- localStorage-ის გამოყენება Cart-ის შესანარჩუნებლად შესაძლებელია; უნივერსალური StorageService ან custom storage hook React-ის CORE-ში სავალდებულო არ არის.
- საერთო Cart და Login მდგომარეობა შეიძლება განთავსდეს Context-ში useState-ით; შვილ კომპონენტებს მონაცემები გადაეცემა props-ით და callback props-ით. გამოიყენება lifting state up.

- React CORE-ში აუცილებელია ფუნქციური კომპონენტები, props და მინიმუმ ერთი props.children კომპოზიცია, map() + სტაბილური key, immutable updates, conditional rendering და კონტროლირებადი ფორმები; სრული UI ერთ App კომპონენტში არ უნდა დაგროვდეს.

A1.5 მონაცემთა მოდელები და localStorage რეესტრი

CORE მონაცემები: Product, ProductsResponse, CartItem/CartState, მარტივი მომხმარებელი (მაგ. username) და Checkout-ისთვის საჭირო ფორმის ველები. FULL: Reviews და გაფართოებული Order მოდელი.

ლოკალური მონაცემებისთვის გამოიყენე მხოლოდ რეალურად საჭირო გასაღებები: CORE-ში კალათა შეიძლება ინახებოდეს shop_cart-ში; სასწავლო Login-ის session persistence არ მოითხოვება. FULL-ში შესაძლებელია shop_orders და shop_theme.

Key	რა ინახება	როდის იწერება
(არ გამოიყენება)	სასწავლო Login-ში არ ინახება ავტორიზაციის მონაცემები	—
shop_user (არასავალდებულო)	მხოლოდ username ან login boolean	სურვილისამებრ; React Context საკმარისია
shop_cart	CartState - კალათის ელემენტები + promo კოდი (აპლიკაციის მთავარი state)	ყოველი ცვლილებისას - reducer-ის შემდეგ, useEffect-ით
shop_orders	Order ობიექტების მასივი	შეკვეთის განთავსებისას
shop_reviews	ობიექტი: productId -> Review[] (FULL)	შეფასების დამატებისას
shop_theme	"light" ან "dark"	თემის გადართვისას

TypeScript მოდელები (types/): CORE-ში გამოიყენება მიმდინარე React ფუნქციების Product, Cart, მარტივი Login და Checkout ტიპები. ქვემოთ მოცემული გაფართოებული მოდელები FULL-ის არჩევითი მაგალითებია.

```
// types/product.ts
export interface Review {
  rating: number; comment: string; date: string; reviewerName: string;
}
export interface Product {
  id: number; title: string; description: string; category: string;
  brand?: string; price: number; discountPercentage: number;
  rating: number; stock: number; thumbnail: string; images: string[];
  reviews: Review[];
}
export interface ProductsResponse {
  products: Product[]; total: number; skip: number; limit: number;
}
export interface Category { slug: string; name: string; url: string; }

// types/user.ts
export interface AuthUser {
  username: string;
  isLoggedIn: boolean;
}
export interface LoginFormValues { username: string; password: string; }




// types/cart.ts
export interface CartItem {
  productId: number; title: string; thumbnail: string; price: number;
  discountPercentage: number; stock: number; quantity: number;
}
export interface CartState { items: CartItem[]; promoCode?: string | null; }

// types/order.ts
export type OrderStatus = "Processing" | "Shipped" | "Delivered"; // FULL
// რეალური გადახდის მეთოდები ამ პროექტში არ გამოიყენება; PaymentMethod საჭირო არ არის.
export interface ShippingInfo {
  fullName: string; email: string; address: string; city: string;

}
export interface Order { // [FULL] შეკვეთის გაფართოებული ლოკალური მოდელი
  id: number;            // Date.now() განთავსებისას
  number: string;        // "10X-00001" - რიგითი, 5 ციფრი
  items: CartItem[]; shipping: ShippingInfo;
  // შეკვეთა იქმნება ლოკალურად; ბარათის მონაცემები/გადახდის მეთოდი არ არსებობს
  subtotal: number; discount: number; shippingCost: number; total: number;
  placedAt: string;      // new Date().toISOString()
}

// types/api.ts - ყველა ასინქრონული მონაცემის ერთი ფორმა
export type AsyncState<T> =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; message: string };
```

A1.6 მონაცემების წყარო - DummyJSON API

პროდუქტისა და კატეგორიების მონაცემები მოდის DummyJSON საჯარო სატესტო API-დან (dummyjson.com). Login გამოიყენებს ლოკალურ controlled form-ს; ავტორიზაციის API ინტეგრაცია არ მოითხოვება.

მეთოდი და მისამართი	რას აბრუნებს	სად
Login: ლოკალური React ფორმა	username/isLoggedIn — Context state	P1 (CORE)
GET /products?limit=12&skip=0&sortBy=price&order=asc	{ products, total, skip, limit }	P2
GET /products/search?q=phone&limit=12&skip=0	იგივე სტრუქტურა - ძებნის შედეგი	P2
GET /products/categories	[{ slug, name, url }]	P2
GET /products/category/{slug}?limit=12&skip=0	იგივე სტრუქტურა - ერთი კატეგორია	P2, P3
GET /products/{id}	Product (reviews-ით) ან 404	P3
Checkout: ლოკალური შეკვეთის სიმულაცია	order summary + confirmation	P5 (CORE)
Profile: ლოკალური რედაქტირება (FULL)	React state	P7 (FULL)

DummyJSON-ის პროდუქტების API გამოიყენება ჩამოტვირთვისთვის. Checkout-ში შეკვეთის გაფორმება ადგილობრივად სიმულირდება და წარმატების დადასტურებას აჩვენებს; რეალური შეკვეთის, გადახდისა და სერვერული ავტორიზაციის განხორციელება არ მოითხოვება.

- **სასწავლო Login-ისთვის სტუდენტს შეუძლია გამოიყენოს ნებისმიერი არაცარიელი username და password (მარტივი ვალიდაციით). სისტემა არ ამოწმებს რეალურ ანგარიშს.**
- **პაგინაცია:** skip = (page - 1) × limit; გვერდების რაოდენობა = Math.ceil(total / limit). API-ს პასუხში პროდუქტების მასივი დევს data.products ველში, ჯამური რაოდენობა - data.total-ში.

A1.7 ტექნიკური შეზღუდვები

- **CORE ტექნოლოგიები:** Vite, React 19, TypeScript (strict), React Router, Tailwind CSS v4, DummyJSON პროდუქტების API. Login და Checkout კეთდება React controlled inputs-ით, useState-ით, onSubmit/preventDefault-ით და მარტივი ვალიდაციით. HTTP: fetch ან axios. Create React App არ გამოიყენება.
- **CORE სტილიზაცია:** აპლიკაციის ძირითადი კომპონენტები კეთდება Tailwind CSS v4 utility კლასებით (Flex/Grid, spacing, typography, responsive, hover/focus). ჩვეულებრივი CSS/SASS დამხმარედ დასაშვებია. **FULL:** @theme, dark mode, custom utilities.
- **React State: useState/useEffect, props/callback props, lifting state up, Context (Cart/Login).**

- **TypeScript:** "strict": true; any აკრძალულია (გამონაკლისი - unknown type guard-ით). npm run typecheck (tsc --noEmit) 0 შეცდომით - ეს ჩაბარების პირობაა, არა რეკომენდაცია.
- **UI ტექსტების ენა:** ინტერფეისი ინგლისურად.
- **Deployment სავალდებულოა:** Vercel ან Netlify;
- **სკრიპტები package.json-ში:** dev, build, preview, typecheck, lint
- **FULL**: ფორმების გაფართოებული მართვა: Formik და Yup გამოიყენება Login ფორმის გაფართოებული რეალიზაციისთვის. Formik მართავს ფორმის მდგომარეობას, ველების ცვლილებებს, შეცდომებსა და გაგზავნას, ხოლო Yup გამოიყენება Validation Schema-ს განსაზღვრისთვის. Formik/Yup არ წარმოადგენს CORE მოთხოვნას. CORE-ში Login და Checkout შეიძლება განხორციელდეს React-ის სტანდარტული controlled inputs-ითა და მარტივი ვალიდაციით.

A2. გვერდების დეტალური სპეციფიკაციები

თითო გვერდს ერთი და იგივე სტრუქტურა აქვს: დიაგრამა, ელემენტების რეესტრი (რა არის და რა ხდება), დეტალური მოთხოვნები, Out of Scope და Measurable Outcome.

P0 - გლობალური წესები [CORE + FULL]

CORE — Layout, React Router, Header (Catalog/Cart/Login), 404, მარტივი Checkout guard (შესულია/არ არის შესული React state-ში), loading/error/empty და სასწავლო შეტყობინებები. FULL — Orders/Profile navigation, Theme, Toast framework, lazy routes, დამატებითი UI მდგომარეობები. ოპტიმიზაციის მოლულის Error Boundary/DEBUG-LOG მოთხოვნები დამოუკიდებლად განისაზღვრება.

P0.1 - Routing და Layout

- ერთი BrowserRouter; ერთი **Layout მარშრუტი** (Header + <Outlet /> + Footer), რომელშიც ყველა გვერდი ჩის - Header ხუთჯერ არ იწერება.
- მიმდინარე გვერდის ბმული ვიზუალურად გამოკვეთილია (NavLink-ის active კლასი).
- * მარშრუტი -> NotFound გვერდი: სათაური „Page not found“, ტექსტი „The page you are looking for does not exist.“, ბმული „Go to catalog“ -> /. იგივე გვერდი ჩანს არარსებულ პროდუქტზე (/products/999999) და არარსებულ შეკვეთაზე.
- მარშრუტის ცვლილებისას გვერდი ზემოდან იწყება (ScrollToTop კომპონენტი: useLocation + useEffect + window.scrollTo).
- [FULL] გვერდების lazy loading React.lazy/Suspense-ით. CORE-ში საკმარისია BrowserRouter, Routes, Link/NavLink, useParams, useNavigate და საერთო Layout.

P0.2 — მარტივი Login State და Checkout-ზე წვდომა [CORE]

- **Checkout-ის გახსნისას აპლიკაცია AuthContext-ის მარტივ boolean/user state-ს ამოწმებს: თუ მომხმარებელი არ არის შესული → /login. Login-ის წარმატების შემდეგ → /checkout. სხვა გვერდები საჯაროა; რეალური ავტორიზაციის დაცვა არ ხორციელდება.**
- **Login გვერდზე მომხმარებელი ავსებს controlled fields-ს; წარმატებული submit ცვლის AuthContext-ის მდგომარეობას. Logout აბრუნებს მას guest მდგომარეობაში.**

- **Refresh-ისას Login-ის მდგომარეობის დაკარგვა დასაშვებია. სესიის შენარჩუნება (მაგალითად, username/isLoggedIn-ის localStorage-ში შენახვით) არჩევითია და CORE მოთხოვნას არ წარმოადგენს.**
- Login/Logout ლოგიკა მოთავსებულია საერთო AuthContext-ში, კომპონენტებს შორის მონაცემების ზედმეტი გადაცემის გარეშე.

P0.3 - Header (ნავიგაცია)

- Header: „10X Shop“, Catalog, Cart და Log in/Log out ბმულები. Cart badge შეიძლება მიიღებოდეს CartContext-იდან. Orders, Profile და Theme მხოლოდ FULL-ში ჩანს.
- **Logout ცვლის AuthContext-ის მომხმარებლის მდგომარეობას guest-ად და აბრუნებს კატალოგში; კალათა ამ მოქმედებით არ უნდა წაიშალოს.**
- მობილურზე (≤ 480px) ბმულები burger მენიუში იკეცება ან მეორე რიგში გადადის - ჰორიზონტალური scroll დაუშვებელია (O3).

P0.4 - თემა

- Dark / Light გადამრთველი: თემის ცვლილება Tailwind-ის გამოყენებით.

P0.5 - შეტყობინებების სტანდარტი

- **CORE ფორმის შეცდომები ჩნდება შესაბამის ველთან და მუშავდება React state-ით; blur-ზე ვალიდაცია და დამატებითი UX — FULL.**
- **[FULL] საერთო Toast framework; CORE-ში დასაშვებია მარტივი შეტყობინება ან inline feedback.**
- **უკეთესი ვარიანტია toast component-ის შექმნა.**

P0.6 - Loading / Error / Empty სტანდარტი

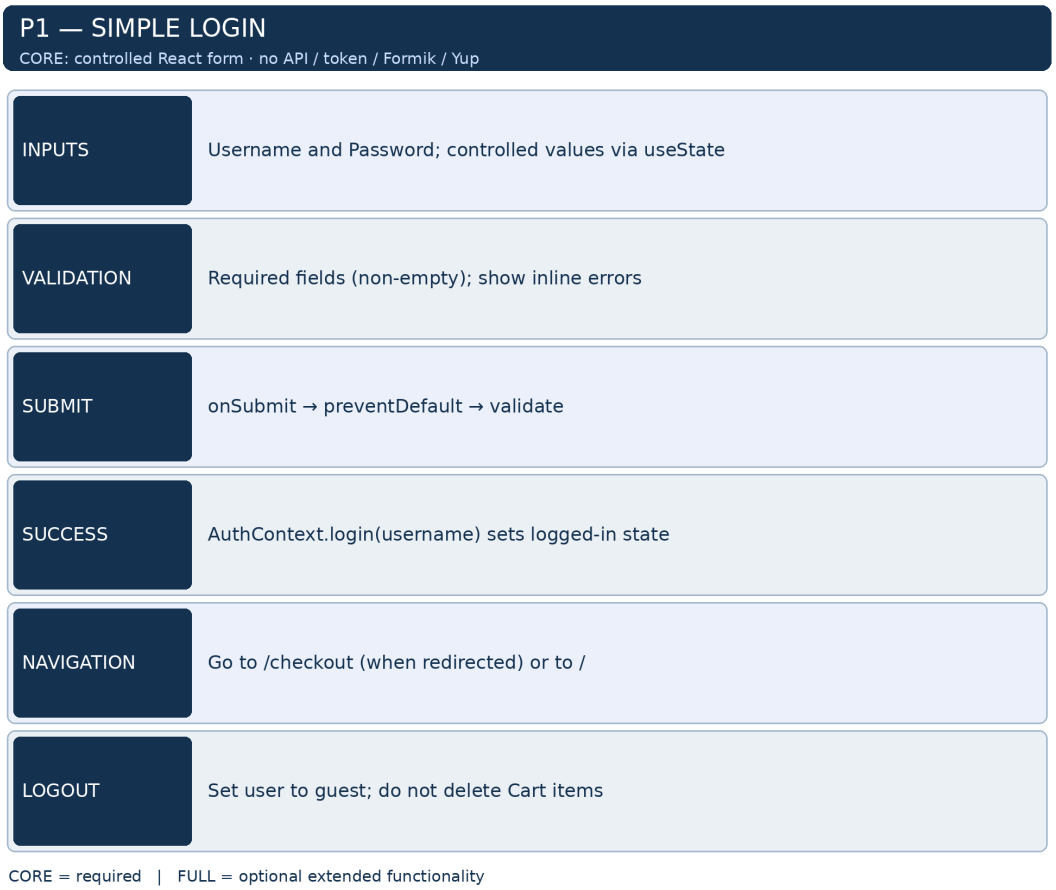
- CORE: API-ის ჩატვირთვაზე ჩანს Loading, შეცდომაზე — Error შეტყობინება, ცარიელ მონაცემებზე — Empty State. მართვა ხდება useState/useEffect-ით და conditional rendering-ით. Skeleton/Retry-ის გაფართოებული ფორმა — FULL.

Header-ის ელემენტების რეესტრი (ყველა გვერდზე)

ელემენტი	რას ემსახურება	რა ხდება დაჭერისას / გამოყენებისას
ლოგო „10X Shop“	ბრენდი + კატალოგზე დაბრუნება	click -> /
ბმული Catalog	ნავიგაცია	click -> /; მიმდინარე გვერდზე active კლასი აქვს
ბმული Cart + badge	კალათაზე გადასვლა; რამდენი ნივთია	click -> /cart; badge = Σ quantity (CartContext); 0-ზე badge იმალება
ბმული Orders	შეკვეთების ისტორია	click -> /orders; ჩანს მხოლოდ ავტორიზებულს
ბმული Profile / ღილაკი Log in	პროფილი ან შესვლა	Login: /login; Profile: მხოლოდ FULL
თემის ღილაკი	Dark / Light გადართვა	click -> data-theme იცვლება -> shop_theme იწერება -> შემდეგ ჩატვირთვაზეც ეს თემაა
Log out	სესიის დახურვა	React Context: isLoggedIn=false; navigate("/")

Measurable Outcome

✔ CORE: Checkout-ის გახსნა სტუმრისთვის იწვევს /login-ზე გადასვლას; მარტივი Login-ის შემდეგ შესაძლებელია Checkout-ზე დაბრუნება.



✔ CORE: Login ცვლის Context state-ს; Logout აბრუნებს სტუმრის მდგომარეობას.

✔ /products/5-ის გადატვირთვა deploy-ზე მუშაობს (SPA rewrite); უცნობი მისამართი 404 გვერდს აჩვენებს

✔ CORE: Header, Cart badge და Login/Logout მუშაობს საერთო React state-ით; loading/error/empty მდგომარეობები გასაგებად ჩანს. FULL: Theme და Error Boundary.

P1 - Login გვერდი (/login) [CORE]

CORE — Username/Password controlled inputs, useState, onChange, onSubmit და preventDefault; მარტივი required ვალიდაცია; AuthContext-ში მარტივი მომხმარებლის მდგომარეობის შენახვა და Checkout-ზე გადასვლა.
FULL — ველების დამატებითი ვიზუალური ვალიდაცია, loading/disabled მდგომარეობები და უფრო დახვეწილი UX. Formik + Yup: Login ფორმის გაფართოებული ვერსია ხორციელდება Formik-ისა და Yup-ის გამოყენებით. ფორმის სამართავად გამოიყენება useFormik Hook, ხოლო ველების ვალიდაციისთვის Yup Validation Schema.

ელემენტების რეესტრი

ელემენტი	რას ემსახურება	რა ხდება დაჭერისას / გამოყენებისას
ველები Username, Password	ავტორიზაციის მონაცემები	Controlled React form: onChange → useState → required validation → Context login → navigate
ღილაკი „Log In“	სისტემაში შესვლა	Controlled React form: onChange → useState → required validation → Context login → navigate
ბმული „← Back to catalog“	უკან დაბრუნება	click -> / (არაფერი ინახება)

P1.1 - განლაგება და ელემენტები

- Core: ცენტრირებული Login ფორმა: Username, Password, „Log In“ ღილაკი და „← Back to catalog“.
- Full: Sign Up და პაროლის აღდგენა

P1.2 — Controlled Form და მარტივი ვალიდაცია [CORE]

ველი	ვალიდაციის წესი	შეცდომის ტექსტი (ზუსტი)
Username	სავალდებულო; trim()-ის შემდეგ არაცარიელი	Username is required
Password	სავალდებულო; მინიმუმ 6 სიმბოლო	Password is required / Password must be at least 6 characters

შეცდომა ჩნდება შესაბამის ველთან: Username is required / Password is required. ფორმა არ იგზავნება, სანამ აუცილებელი ველები ცარიელია.

P1.3 — Login-ის ქცევა [CORE]

1. მომხმარებელი ავსებს input-ებს; მათი value დაკავშირებულია useState-თან. onSubmit იძახებს preventDefault()-ს და ამოწმებს ველებს.

2. ვალიდური ფორმისას გამოიძახება AuthContext-ის login(username) ფუნქცია, რომელიც აღგენს isLoggedIn=true და ინახავს username-ს React state-ში.

3. მომხმარებელი გადადის /checkout-ზე, თუ Login იქიდან გაიხსნა, სხვა შემთხვევაში — კატალოგზე. ეს მოქმედება სრულდება useNavigate-ით.

4. Logout ასუფთავებს Login-ის React state-ს. რეალური credentials არც ინახება და არც იგზავნება სერვერზე.

[FULL] დამატებითი success/error feedback და disabled მდგომარეობა.

Measurable Outcome

✔ controlled ფორმა: Username/Password, required validation, submit without reload, Context state-ის განახლება.

✔ ცარიელ Username/Password ველზე ჩანს შესაბამისი შეცდომა; ფორმა არ სრულდება.

✔ წარმატებული Login-ის შემდეგ Checkout იხსნება; Logout-ის შემდეგ მომხმარებელი სტუმარია.

FULL Measurable Outcome

✔ formik და yup ბიბლიოთეკები დაინსტალირებულია და რეალურად გამოიყენება Login ფორმაში.

✔ useFormik მართავს Username და Password ველებს.

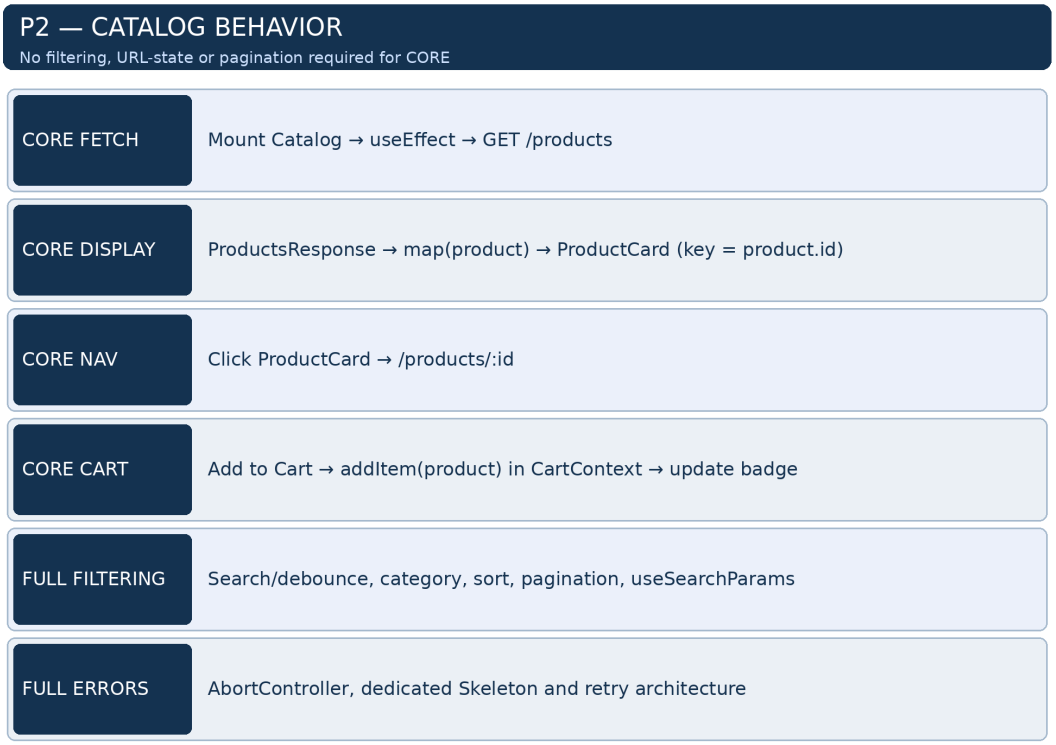
✔ Yup Validation Schema ამოწმებს სავალდებულო ველებსა და Password-ის მინიმალურ სიგრძეს.

✔ Login-ის შემდეგ ნავიგაცია და Logout ფუნქციონალი მუშაობს CORE მოთხოვნების შესაბამისად.

P2 - Catalog (/) [ბირთვი CORE; ძებნა/კატეგორია/სორტი/პაგინაცია/URL-state FULL]



CORE — GET /products, ტიპიზებული მონაცემების ჩამოტვირთვა, ProductCard, პროდუქტების ჩვენება, Product Details-ზე გადასვლა და Add to Cart.



FULL — Search/debounce, categories, sorting, pagination, URL-state, Skeleton/Retry და გაფართოებული მდგომარეობები.

ელემენტების რეესტრი (①-⑦)

ელემენტი	რას ემსახურება	რა ხდება დაჭერისას / გამოყენებისას
① **Search** ველი	ძებნა სახელით	input -> useDebounce(400 ms) -> URL ?q=...&page=1 (category იშლება) -> GET /products/search?q=... -> render. ცარიელი ველი სრულ სიას აბრუნებს
② **Category select**	კატეგორიით გაფილტვრა	change -> URL ?category=slug&page=1 (q იშლება) -> GET /products/category/{slug} -> render. „All categories“ ფილტრს ხსნის
③ **Sort select**	სიის დალაგება	change -> URL ?sort=price-asc&page=1 -> იმავე endpoint-ს ემატება sortBy & order -> render. ვარიანტები: Default / Price: low to high / Price: high to low / Rating: high to low / Name A→Z
④ **ბარათზე დაჭერა**	დეტალების ნახვა	click (ღილაკის გარდა) -> navigate(''/products/'' + id)
⑤ **Add to cart (ბარათზე)**	სწრაფი დამატება	click -> stock > 0 და კალათაში < stock? -> dispatch ADD_ITEM { product, qty: 1 } -> badge ახლდება -> toast „Added to cart ✓“; stock = 0 -> ღილაკი disabled „Out of stock“

ელემენტი	რას ემსახურება	რა ხდება დაჭერისას / გამოყენებისას
⑥ პაგინაცია	გვერდებზე გადასვლა	click -> URL ?page=N -> GET ...&limit=12&skip=(N-1)×12 -> render -> window.scrollTo(0, 0). ‹ და › კიდეებზე disabled
⑦ Retry (შეცდომისას)	ჩატვირთვის გამეორება	click -> იგივე რექვესთი თავიდან -> loading -> render

P2.1 - განლაგება

- ზემოდან ქვემოთ: სათაური „Catalog“ + შედეგების რაოდენობა; Toolbar (ძებნის ველი, კატეგორიის select, სორტის select); პროდუქტების ბადე (desktop - 4 სვეტი, tablet - 2-3, mobile - 1); პაგინაცია; ტექსტი „Showing 13–24 of 194“.

P2.2 - ჩატვირთვა [CORE]

დონის განმარტება: API-დან ProductCard-ების გამოტანა CORE-ია; Skeleton, Retry, exact URL pagination და custom hook-ის მკაცრი არქიტექტურა — FULL.

- [CORE] გვერდის გახსნისას useEffect-ით იტვირთება GET https://dummyjson.com/products?limit=12&skip=0 და მიღებული პროდუქტები ProductCard-ებად გამოისახება. მონაცემების ჩამოტვირთვა შეიძლება პირდაპირ fetch-ით ან ცალკე API ფუნქციით; custom hook სავალდებულო არ არის.
- [CORE] მოთხოვნის მიმდინარეობისას ჩანს მარტივი Loading შეტყობინება; [FULL] Skeleton ბარათები და უფრო დახვეწილი ჩატვირთვის ინდიკატორები.
- **[CORE] API-ის შეცდომისას ჩანს გასაგები Error შეტყობინება, ხოლო ცარიელი მონაცემებისას — Empty State. [FULL] საერთო ApiClient აბსტრაქცია, Retry ღილაკი და გაფართოებული შეცდომების დამუშავება.**
- **[FULL] AbortController:** თუ პარამეტრი შეიცვალა, სანამ წინა პასუხი მოვიდოდა, წინა რექვესთი უქმდება (useEffect-ის cleanup-ში controller.abort()) - ასე ძველი პასუხი ახალს ვერ „გადაფარავს“. დაცვაზე: „რა მოხდება ამის გარეშე?“

P2.3 - ბარათის რენდერი [CORE]

დონის განმარტება: პროდუქტის ტიპიზებული ბარათები CORE-ია; Related-ში იმავე კომპონენტის ხელახალი გამოყენება, ფასებისა და stock-ის ყველა ზუსტი მდგომარეობა — FULL.

- ერთი კომპონენტი ProductCard (props: { product: Product; onAdd: (p: Product) => void }); ბადე - ProductGrid, რომელიც map-ით ბარათებს ხატავს (key = product.id). იგივე ProductCard გამოიყენება P3-ის related ბლოკში - ეს კომპოზიციის მთავარი მტკიცებულებაა.
- ბარათზე ჩანს: thumbnail (loading="lazy", width და height ატრიბუტებით - CLS-ის წინააღმდეგ), სათაური, კატეგორია · ბრენდი, **ფასი:** ფასდაკლებული ფასი მსხვილად + საწყისი ფასი გადახაზული + მწვანე ბეჯი „-9%“ (მხოლოდ თუ discountPercentage ≥ 1), რეიტინგი „★ 4.7“, **მარაგის ბეჯი:** „In stock“ / „Only N left!“ (stock ≤ 5, ნარინჯისფერი) / „Out of stock“ (stock = 0, წითელი), ღილაკი „Add to cart“.
- ფასდაკლებული ფასი = price × (1 - discountPercentage / 100), 2 ათწილადამდე - ერთი ფუნქცია discountedPrice() utils-ში; ფორმატი formatPrice()-ით: $1,299.00.
- ცარიელი შედეგისას: No products found. + ბმული „Clear filters“, რომელიც URL-ს ასუფთავებს.

P2.4 - ძებნა [FULL]

- input-ში აკრეფისას ლოკალური state ეგრევე იცვლება (ველი არ „ჭედავს“), URL-ის ?q კი 400 ms დაყოვნებით - custom hook useDebounce(value, 400). 5 ასოს აკრეფაზე 1 რექვესთი მიდის, არა 5.
- q არსებობისას endpoint-ია /products/search?q=...; ძებნა category-ს ასუფთავებს (API-ს შეზღუდვა, იხ. A1.6) და page-ს 1-ზე აბრუნებს.

P2.5 - კატეგორიის ფილტრი [FULL]

- ჩატვირთვისას GET /products/categories -> select-ში „All categories“ + კატეგორიების name-ები (value = slug). არჩევისას URL ?category=slug&page=1, q იშლება; endpoint - /products/category/{slug}.

P2.6 - სორტი [FULL]

- select, 5 ვარიანტი: Default (value ""), Price: low to high (price-asc), Price: high to low (price-desc), Rating: high to low (rating-desc), Name A→Z (title-asc). URL ?sort=price-asc -> API-ს პარამეტრები sortBy=price&order=asc ემატება მიმდინარე endpoint-ს.
- თუ რომელიმე endpoint-ი სორტის პარამეტრს უგულებელყოფს - დააფიქსირე DEBUG-LOG-ში და მიმდინარე გვერდი client-side დაასორტე (ახსნა შეძლო, რატომ არის ეს კომპრომისი და არა გადაწყვეტა).

P2.7 - პაგინაცია [FULL]

- limit = 12; skip = (page - 1) × 12; გვერდების რაოდენობა = Math.ceil(total / 12). კომპონენტი Pagination (props: page, totalPages, onChange): ‹ 1 2 3 ... N › - მიმდინარე გამოკვეთილი, ‹ და › კიდეებზე disabled. გვერდის ცვლილება სხვა პარამეტრებს ინარჩუნებს.

P2.8 - URL როგორც state-ის წყარო [FULL]

- ოთხივე პარამეტრი (q, category, sort, page) ცხოვრობს URL-ში (useSearchParams), არა useState-ში: კომპონენტი კითხულობს URL-ს -> აწყობს რექვესთს -> ხატავს. ამიტომ refresh, Back/Forward და ბმულის გაზიარება თავისთავად მუშაობს - ერთი წყარო, ერთი ჭეშმარიტება.
- ყველა ფილტრი კომბინირებადია ამ წესებით: ძებნა და კატეგორია ურთიერთგამომრიცხავია; სორტი და პაგინაცია ნებისმიერთან მუშაობს; ფილტრის ცვლილება page-ს 1-ზე აბრუნებს.

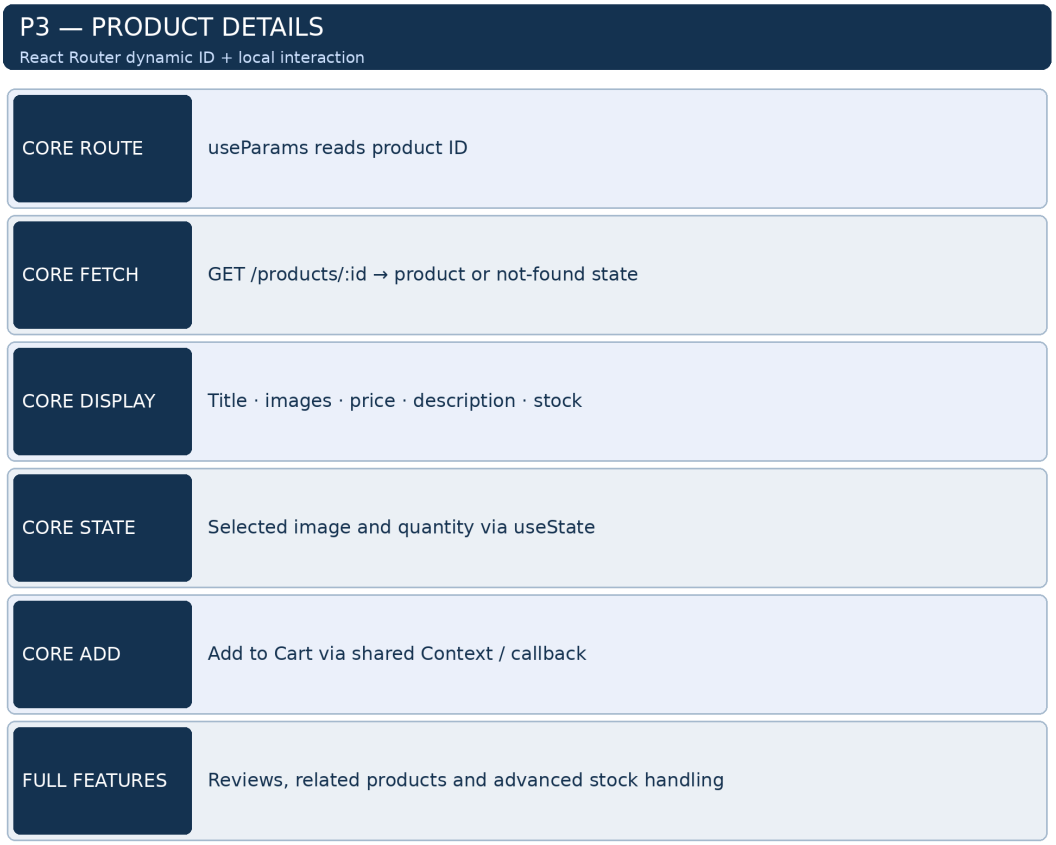
Out of Scope

- Infinite scroll (ბონუსი); ფასის დიაპაზონის ფილტრი (ბონუსი); Wishlist (ბონუსი); grid / list ხედის გადამრთველი (ბონუსი); ძებნის ისტორია.

Measurable Outcome

✔ [CORE] Catalog-ის გახსნისას DummyJSON-დან პროდუქტები იტვირთება და ProductCard კომპონენტებით სწორად გამოისახება.

✔ [CORE] პროდუქტების სია იყენებს map()-ს და სტაბილურ key-ს; პროდუქტის ბარათიდან შესაძლებელია დეტალურ გვერდზე გადასვლა და კალათაში დამატება.



✔ [FULL] Search/debounce, Category, Sorting, Pagination და URL-state მუშაობს დოკუმენტში მითითებული დამატებითი წესებით, თუ სტუდენტი ამ ფუნქციებს ახორციელებს.

✔ [CORE] API-ის ჩატვირთვის, შეცდომისა და ცარიელი შედეგის მარტივი მდგომარეობები ჩანს. [FULL] Skeleton, Retry და გაფართოებული შეცდომის დამუშავება.

P3 - Product (/products/:id) [ბირთვი CORE; Reviews/Related FULL]

CORE — /products/:id API, პროდუქტის დეტალები, სურათი/ფასი/მარაგის ძირითადი ასახვა და Add to Cart.
FULL — სრულყოფილი გალერეა და stock edge cases, Reviews, Related products და დამატებითი კომპონენტური UX.

ელემენტების რეესტრი

ელემენტი	რას ემსახურება	რა ხდება დაჭერისას / გამოყენებისას
ბმული „← Back to catalog“	უკან დაბრუნება	click -> navigate(-1) (ფილტრები ადგილზეა, რადგან URL-შია)
გალერეა	ფოტოების ნახვა	thumbnail-ზე click -> მთავარი ფოტო იცვლება (ლოკალური useState)
რაოდენობის stepper - / +	რამდენი ცალი	click -> qty ± 1; საზღვრები 1 ≤ qty ≤ stock; კიდეებზე ღილაკი disabled
ღილაკი „Add to cart“	კალათაში დამატება	click → CartContext.addItem(product, qty) → immutable state update → კალათის განახლება; [FULL] reducer/Toast და რთული stock შემოწმება.

ელემენტი	რას ემსახურება	რა ხდება დაჭერისას / გამოყენებისას
ტაბები Description / Reviews	ინფორმაციის დაჯგუფება	click -> აქტიური ტაბი (useState); Reviews-ის სათაურში რაოდენობა ჩანს
Write a review (FULL)	შეფასების დამატება	controlled React form -> ვალიდაცია (P3.4) -> Review ობიექტი -> shop_reviews -> სია და საშუალო რეიტინგი ახლდება -> toast „Review added ✓“
Related products (FULL)	მსგავსი პროდუქტები	GET /products/category/{slug}?limit=5 -> მიმდინარეს გარდა პირველი 4 -> იგივე ProductCard

P3.1 - განლაგება

- Desktop - ორი სვეტი (გალერეა | ინფორმაცია), mobile - ერთი. ინფორმაციის ბლოკი: სათაური, ბრენდი · კატეგორია (კატეგორია - Link -> /?category=slug), ფასის ბლოკი (როგორც ბარათზე, უფრო დიდი), „★ 4.7 · 12 reviews“, მარაგის სტატუსი, აღწერა, stepper + „Add to cart“, ტაბები.

P3.2 - ჩატვირთვა [CORE]

დონის განმარტება: პროდუქტის API-დან ჩვენება CORE-ია; სრული გალერეა/რთული 404 და edge-case დამუშავება — FULL.

- useParams() -> id. თუ Number(id) არ არის მთელი დადებითი რიცხვი (/products/abc) -> NotFound რექვესთის გარეშე.
- GET /products/{id} -> loading: Skeleton; 404 (ApiError.status === 404) -> NotFound გვერდი; სხვა შეცდომა -> ტექსტი + Retry; წარმატება -> რენდერი. აპლიკაცია არც ერთ შემთხვევაში არ „ტყდება“.
- id-ის ცვლილება (related-დან სხვა პროდუქტზე გადასვლა) ახალ რექვესთს იწვევს - useEffect-ის dependency-ში id უნდა იყოს. ეს კლასიკური ბაგია: გვერდი იცვლება, მონაცემი - არა.

P3.3 - მარაგი და „Add to cart“ [CORE]

- stock > 5 -> „In stock“; 1 ≤ stock ≤ 5 -> „Only {stock} left!“ (ნარინჯისფერი); stock = 0 -> „Out of stock“ (წითელი), stepper და ღილაკი disabled.
- [CORE] კალათაში დამატება ხდება React state/CartContext-ის addItem ფუნქციით და immutable update-ით. არსებული რაოდენობისა და მარაგის გათვალისწინებით ზედმეტი ნივთები არ ემატება; შედეგი ჩანს მარტივი inline შეტყობინებით. [FULL] reducer-ის action-ები, საერთო Toast სისტემა და მარაგის რთული edge-case-ები.

P3.4 - შეფასებები [FULL]

- სია: ჯერ ლოკალური შეფასებები (shop_reviews[id], ახალი ზემოთ), შემდეგ API-ს product.reviews. თითო: reviewerName, ვარსკვლავები, თარიღი (toLocaleDateString), კომენტარი. საშუალო რეიტინგი ითვლება ორივედან - useMemo-თი.

ველი	ვალიდაციის წესი	შეცდომის ტექსტი (ზუსტი)
Rating	სავალდებულო; 1-5 (ვარსკვლავები ან select)	Please select a rating

ველი	ვალიდაციის წესი	შეცდომის ტექსტი (ზუსტი)
Comment	სავალდებულო; trim()-ის შემდეგ მინიმუმ 10 სიმბოლო	Comment must be at least 10 characters

- წარმატებისას: { rating, comment, date: new Date().toISOString(), reviewerName: ავტორიზებულის firstName + lastName ან „Guest“ } -> shop_reviews[productId]-ის თავში -> ფორმა სუფთავდება -> toast: Review added ✓

P3.5 - Related products [FULL]

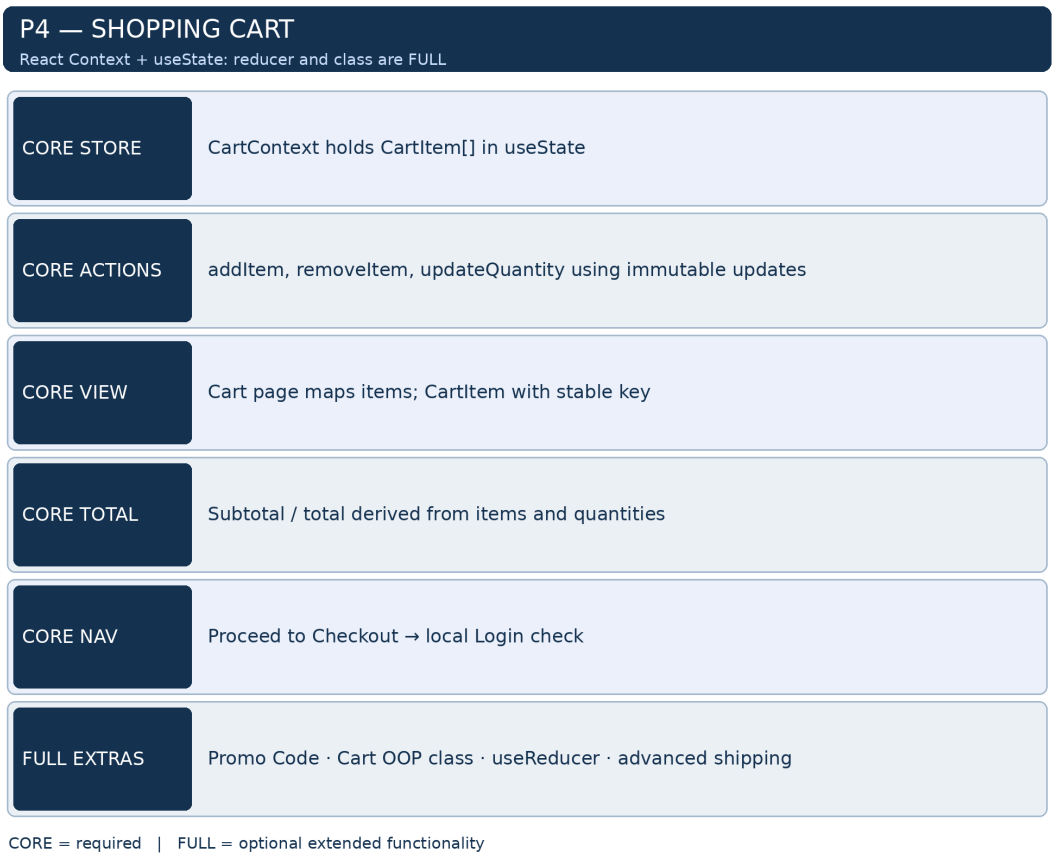
- GET /products/category/{product.category}?limit=5 -> filter(p => p.id !== product.id) -> slice(0, 4) -> ProductGrid იმავე ProductCard-ით. ბარათზე დაჭერა იმავე გვერდზე ახალი id-ით გადადის (P3.2-ის dependency ტესტი).

Out of Scope

- ფოტოს zoom; გაზიარება; პროდუქტების შედარება; შეფასების რედაქტირება / წაშლა.

Measurable Outcome

✔ /products/1 იტვირთება, ფასდაკლებული ფასი ემთხვევა price × (1 – discountPercentage / 100)-ს; stock-ის სამივე მდგომარეობა სწორ ტექსტს აჩვენებს



✔ /products/999999 და /products/abc NotFound გვერდს აჩვენებს – კონსოლში გაუმართავი შეცდომა არ არის

✔ მარაგზე მეტის დამატება შეუძლებელია ზუსტი toast-ით; related-დან გადასვლისას მონაცემი ახლდება (FULL)

✔ შეფასება თარიღით ემატება, გადატვირთვის შემდეგ ადგილზეა და საშუალო რეიტინგს ცვლის (FULL)

P4 - Cart (/cart) [ბირთვი CORE; reducer/OOP/promo FULL]

CORE — კალათის დამატება/რაოდენობის შეცვლა/წაშლა/ჯამის დათვლა, UI-ის სინქრონიზაცია React state-თან.
FULL — Reducer-ის მოწინავე action-ები, Cart კლასი, StorageService<T>, promo code და stock-ის რთული სცენარები.

ელემენტების რეესტრი

ელემენტი	რას ემსახურება	რა ხდება დაჭერისას / გამოყენებისას
① **stepper − / + (ხაზზე)**	რაოდენობის შეცვლა	click -> dispatch SET_QTY { productId, qty } -> 1 ≤ qty ≤ stock -> ჯამები და badge ახლდება -> shop_cart
② **Remove**	ხაზის წაშლა	click -> dispatch REMOVE_ITEM { productId } -> toast „Removed from cart“
③ **Clear cart**	კალათის განულება	click -> confirm("Clear the cart? This cannot be undone.") -> dispatch CLEAR
④ **Promo code + Apply (FULL)**	ფასდაკლება	click -> code === "10X10" (case-insensitive)? -> dispatch APPLY_PROMO -> „Discount (10X10 -10%)“ + „Remove“ ბმული; სხვა -> ველის შეცდომა „Invalid promo code“
⑤ **Proceed to checkout**	შეკვეთაზე გადასვლა	click -> navigate("/checkout"); სტუმარს ProtectedRoute /login-ზე გადაიყვანს (from = /checkout) - მეორე შემოწმებას აქ არ წერ
⑥ **Browse products (ცარიელი კალათა)**	კატალოგში დაბრუნება	click -> /

P4.1 - განლაგება

- მარცხნივ - ხაზების სია: thumbnail, სათაური (Link -> /products/:id), ერთეულის ფასდაკლებული ფასი, stepper, ხაზის ჯამი, Remove; სიის ქვეშ „Clear cart“. მარჯვნივ - შეჯამების ბარათი: Subtotal, Discount, Shipping, **Total** + ღილაკი „Proceed to checkout“ (მობილურზე - სიის ქვეშ).
- ცარიელი კალათა: „Your cart is empty.“ + ბმული „Browse products“; შეჯამება არ ჩანს.

P4.2 - Cart State და მოქმედებები [CORE]; reducer-ის დეტალური არქიტექტურა [FULL]

დონის განმარტება: კალათის რეალური მოქმედებები CORE-ია. ქვემოთ მოცემული reducer/action-ების ზუსტი კონსტრუქცია წარმოადგენს FULL იმპლემენტაციის ნიმუშს.

- state: { items: CartItem[]; promoCode: string | null }. მოქმედებები - discriminated union (ქვემოთ); reducer-ი **სუფთა ფუნქციაა** - localStorage-ს არ ეხება. შენახვა CartProvider-ის useEffect-შია: state შეიცვალა -> shop_cart-ში ჩაიწერა. საწყისი state - useReducer-ის მესამე არგუმენტით (lazy init) shop_cart-დან.

- წესები: ADD_ITEM – თუ productId უკვე არის, quantity += qty, თუ არა – push; ორივე შემთხვევაში შედეგი ≤ stock, თორემ state არ იცვლება და კომპონენტი toast-ს აჩვენებს; SET_QTY – 1 ≤ qty ≤ stock; REMOVE_ITEM – filter; CLEAR – items: [], promoCode: null; APPLY_PROMO / REMOVE_PROMO.

```
// context/cart.reducer.ts
export type CartAction =
  | { type: "ADD_ITEM"; product: Product; qty: number }
  | { type: "SET_QTY"; productId: number; qty: number }
  | { type: "REMOVE_ITEM"; productId: number }
  | { type: "CLEAR" }
  | { type: "APPLY_PROMO"; code: string }
  | { type: "REMOVE_PROMO" };

export function cartReducer(
  state: CartState, action: CartAction
): CartState {
  switch (action.type) {
    case "ADD_ITEM": { /* ... */ }
    // ...
    default: {
      const _exhaustive: never = action;   // ყველა ტიპი დაფარულია
      return state;
    }
  }
}
```

P4.3 – ჯამის გამოთვლა [CORE]; Cart კლასი [FULL]

დონის განმარტება: subtotal/total-ის დათვლა CORE-ია; ცალკე Cart კლასში მთელი ბიზნეს-ლოგიკის განთავსება — FULL.

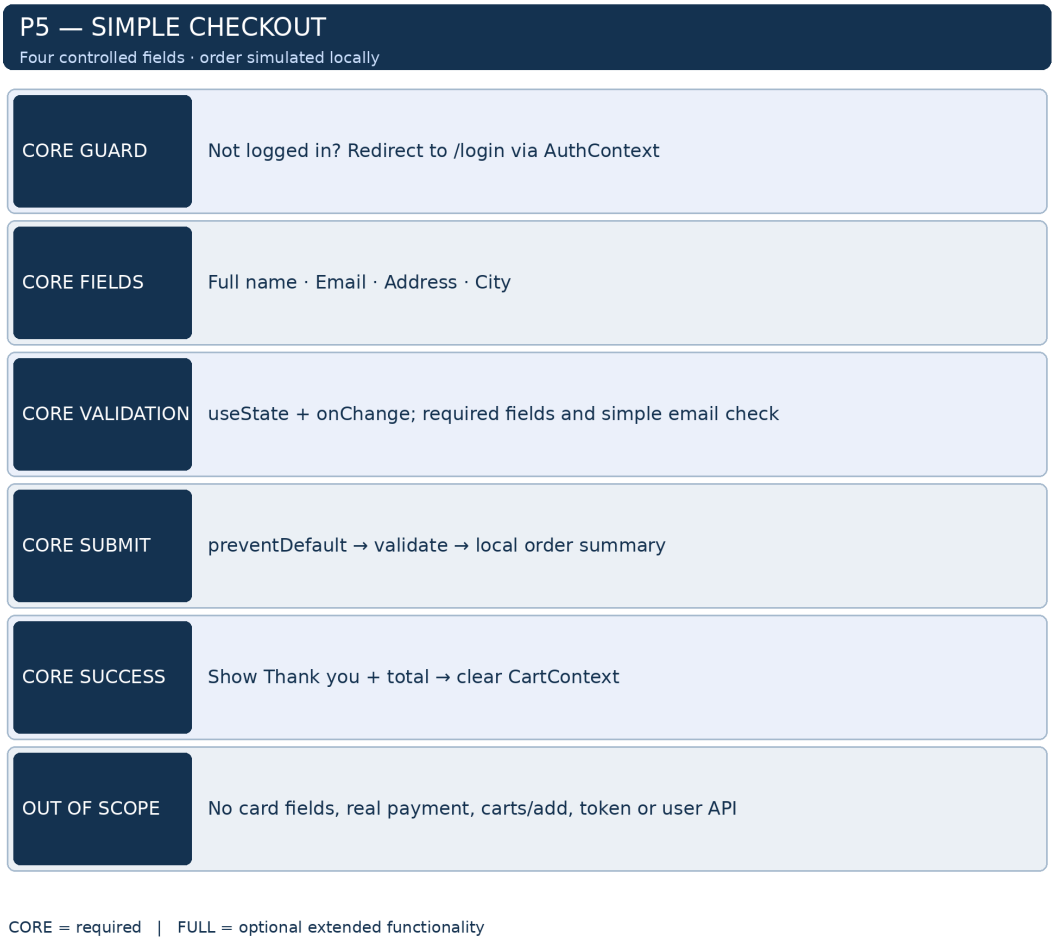
- [FULL] Cart კლასი შეიძლება აერთიანებდეს ფასების გაფართოებულ გამოთვლას; CORE-ში ჯამი მიიღება CartItem[]-იდან ჩვეულებრივი ფუნქციით ან გამოთვლილი მნიშვნელობით.
- **CORE: subtotal/total = პროდუქტების ფასების × quantity ჯამი (ფასდაკლების გათვალისწინებით, თუ ბარათში ჩანს). FULL: promo, shipping thresholds და დამატებითი გამოთვლები.**

P4.4 – Promo კოდი [FULL]

- ველი + „Apply“: მოქმედი კოდი 10X10 (რეგისტრის გარეშე) -> 10% ფასდაკლება, შეჯამებაში ხაზი „Discount (10X10 –10%)“ და „Remove“; არასწორი -> ველის ქვეშ „Invalid promo code“. კოდი shop_cart-ში ინახება და გადატვირთვას უძლებს.

Out of Scope

- „Save for later“; რამდენიმე promo კოდი ერთდროულად; საჩუქრის შეფუთვა; მიწოდების მეთოდის არჩევა.



Measurable Outcome

✔ CORE: კალათაში დამატება, რაოდენობის შეცვლა, წაშლა და ჯამი სწორად მუშაობს; state განახლდება გვერდის reload-ის გარეშე.

✔ CORE: რაოდენობა დადებითი მთელი რიცხვია; დამატებითი stock შეზღუდვები — FULL.

✔ CORE: კალათაში არსებული ნივთების ფასი და რაოდენობა სწორად ქმნის Total-ს; promo/shipping გამოთვლები — FULL.

✔ promo 10X10 მუშაობს, „abc“ აჩვენებს „Invalid promo code“-ს (FULL)

P5 - Checkout (/checkout) [ბირთვი CORE; მოწინავე ვალიდაცია FULL]

CORE — მარტივი Checkout controlled form: fullName, email, address, city; required ვალიდაცია; კალათის ჯამის ჩვენება და ადგილობრივი შეკვეთის დადასტურება.
FULL — დამატებითი shipping ველები, საკონტაქტო ველების გაფართოება, UI მდგომარეობები და შეკვეთის ისტორია. რთული საბანკო/გადახდის ფორმა და რეალური გადახდა არ მოითხოვება.

ელემენტების რეესტრი

ელემენტი	რას ემსახურება	რა ხდება დაჭერისას / გამოყენებისას
ფორმა (3 სექცია, 11 ველი)	მიწოდებისა და გადახდის მონაცემები	[FULL / ამოღებული ძველი სქემა] CORE Checkout: Full name, Email, Address, City — controlled inputs; required validation.
[FULL / ამოღებული ძველი სქემა] CORE Checkout: Full name, Email, Address, City — controlled inputs; required validation.	გადახდის ტიპი	[FULL / ამოღებული ძველი სქემა] CORE Checkout: Full name, Email, Address, City — controlled inputs; required validation.
Order summary	რას იხდის	ავტომატური: იგივე Cart კლასი, რაც P4-ში - ნივთები, Subtotal, Discount, Shipping, Total
ღილაკი „Place order“	შეკვეთის განთავსება	Checkout-ის შეკვეთა ადგილობრივად სიმულირდება React state-ით; POST /carts/add არ გამოიყენება.

P5.1 - განლაგება და დაცვა

- CORE: Checkout ფორმა (Full name, Email, Address, City), Cart summary, „Place order“ ღილაკი; mobile-ზე განლაგება responsive-ია.
- ცარიელი კალათით Checkout-ის გახსნისას მომხმარებელი ბრუნდება /cart-ზე და ხედავს გასაგებ შეტყობინებას.
- საწყისი ფორმის ველები შეიძლება შეივსოს AuthContext-ის username-ით; ყველა სხვა მნიშვნელობა კონტროლდება useState-ით.

P5.2 — Controlled Inputs და მარტივი ვალიდაცია [CORE]

ველი	ვალიდაციის წესი	შეცდომის ტექსტი (ზუსტი)
Full name	სავალდებულო; trim()-ის შემდეგ მინიმუმ 3 სიმბოლო	Full name must be at least 3 characters
Email	[FULL / ამოღებული ძველი სქემა] CORE Checkout: Full name, Email, Address, City — controlled inputs; required validation.	Please enter a valid email address
Phone	სავალდებულო; 9–15 ციფრი, დასაშვებია წამყვანი + და ჰარები	Please enter a valid phone number
Address	სავალდებულო; მინიმუმ 5 სიმბოლო	Address must be at least 5 characters
City	სავალდებულო	City is required
Postal code	სავალდებულო; 4–10 ციფრი	Postal code must be 4–10 digits
Country	სავალდებულო; select (მინ. 5 ქვეყანა, Georgia default-ში)	Please select a country

ველი	ვალიდაციის წესი	შეცდომის ტექსტი (ზუსტი)
[FULL / ამოღებული ძველი სქემა] CORE Checkout: Full name, Email, Address, City — controlled inputs; required validation.	სავალდებულო; "card" ან "cash"	[FULL / ამოღებული ძველი სქემა] CORE Checkout: Full name, Email, Address, City — controlled inputs; required validation.
[FULL / ამოღებული ძველი სქემა] CORE Checkout: Full name, Email, Address, City — controlled inputs; required validation.	მხოლოდ card-ზე: ზუსტად 16 ციფრი (ჰარები იგნორირდება)	[FULL / ამოღებული ძველი სქემა] CORE Checkout: Full name, Email, Address, City — controlled inputs; required validation.
[FULL / ამოღებული ძველი სქემა] CORE Checkout: Full name, Email, Address, City — controlled inputs; required validation.	მხოლოდ card-ზე: ფორმატი MM/YY და მომავალში	[FULL / ამოღებული ძველი სქემა] CORE Checkout: Full name, Email, Address, City — controlled inputs; required validation.

ველი	ვალიდაციის წესი	შეცდომის ტექსტი (ზუსტი)
[FULL / ამოღებული ძველი სქემა] CORE Checkout: Full name, Email, Address, City — controlled inputs; required validation.	მხოლოდ card-ზე: ზუსტად 3 ციფრი	[FULL / ამოღებული ძველი სქემა] CORE Checkout: Full name, Email, Address, City — controlled inputs; required validation.

CORE: input-ების value/onChange მართავს React state; onSubmit იყენებს preventDefault-ს; fullName, email, address და city სავალდებულოა. email-ის მარტივი ფორმატის შემოწმება დასაშვებია. შეცდომები ჩანს შესაბამის ველებთან.

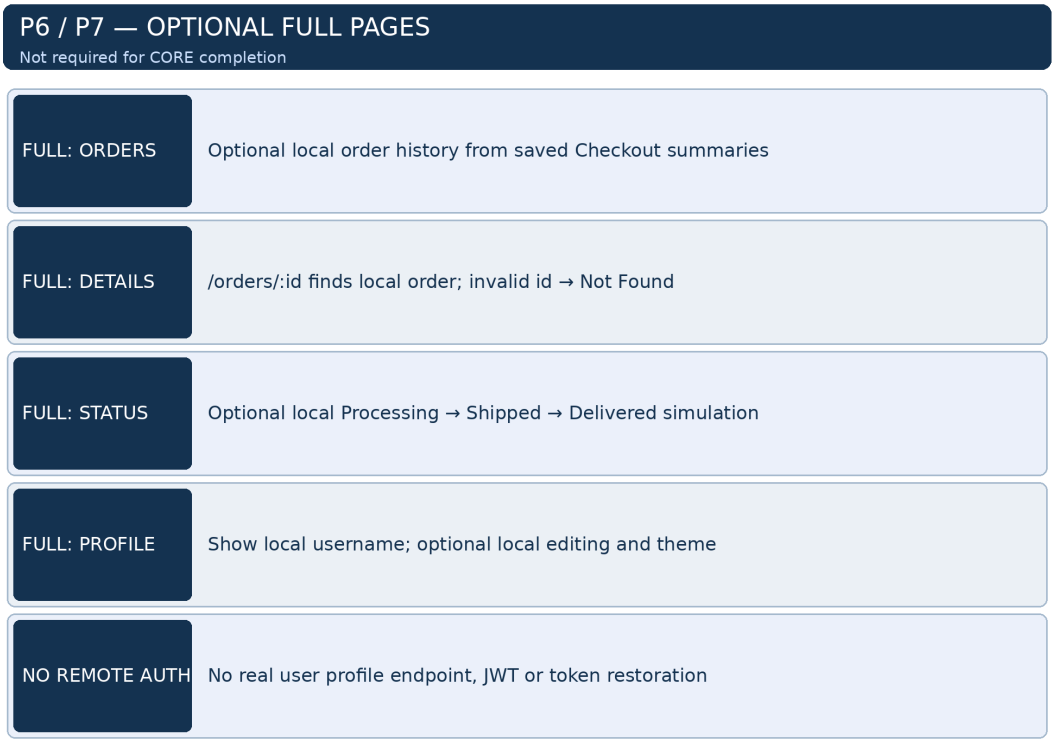
- ცარიელი ან არავალიდური ფორმა არ ასრულებს შეკვეთას; სრულდება მხოლოდ ვალიდური submit. [FULL] დამატებითი ველები და UX.
- საბანკო ბარათის ველები, პირობითი გადახდის წესები და საბანკო მონაცემების დამუშავება არ შედის დავალების მოთხოვნებში.

P5.3 — შეკვეთის გაფორმება [CORE]

1. ვალიდური submit-ისას „Place order“ დროებით disabled ხდება, რათა ორმაგი დაჭერა თავიდან ავიცილოთ.
2. კალათის მიმდინარე ნივთებისა და ფორმის მონაცემების საფუძველზე React state-ში იქმნება მარტივი order summary (არ იგზავნება სერვერზე).
3. ეკრანზე ჩნდება „Thank you! Your order has been placed.“ და შეკვეთის ჯამი.
4. კალათა სუფთავდება; CartContext-ის badge და UI მაშინვე ახლდება.
5. [FULL] სურვილისამებრ შეკვეთა შეიძლება შენარჩუნდეს localStorage-ში და გამოჩნდეს Orders გვერდზე.
6. [FULL] შეუძლია დაემატოს დამუშავების მდგომარეობები, თუმცა საბაზისო შეკვეთა არ მოითხოვს Orders გვერდს.
7. შეცდომის შემთხვევაში inline მესიჯი ჩანს; ფორმის state არ იკარგება.

Out of Scope

- რეალური გადახდა; მისამართების წიგნი (ბონუსი); მრავალსაფეხურიანი checkout (ბონუსი); Luhn-ის ალგორითმი ბარათის ნომერზე (ბონუსი).



CORE = required | FULL = optional extended functionality

Measurable Outcome

✔ CORE: ოთხველიანი controlled form, required errors, სწორი submit preventDefault-ით.

✔ CORE: კალათის ჯამი ჩანს; შეკვეთის დასრულებისას „Thank you!“ დადასტურება და კალათის გასუფთავება მუშაობს.

✔ FULL: Order history/localStorage სურვილისამებრ; ბარათის მონაცემები არ გამოიყენება.

✔ CORE: UI უცვლელად მუშაობს სხვადასხვა ეკრანზე და submit არ იწვევს გვერდის გადატვირთვას.

P6 - Orders გვერდი (/orders, /orders/:id) [FULL]

CORE — სავალდებულო მოთხოვნა არ არის.
FULL — Orders სია, დეტალური გვერდი, სტატუსის სიმულაცია და Reorder.

ელემენტების რეესტრი

ელემენტი	რას ემსახურება	რა ხდება დაჭერისას / გამოყენებისას
შეკვეთის ბარათი (სიაში)	დეტალების ნახვა	click -> /orders/:id

ელემენტი	რას ემსახურება	რა ხდება დაჭერისას / გამოყენებისას
სტატუსის ბეჯი	სად არის შეკვეთა	ავტომატური: ითვლება placedAt-დან (P6.2); Processing - ნაცრისფერი, Shipped - ლურჯი, Delivered - მწვანე
Reorder (დეტალებში)	იგივე ნივთების ხელახლა შეკვეთა	click -> ყოველ ნივთზე dispatch ADD_ITEM -> toast „Items added to cart ✓“ -> /cart
Browse products (ცარიელი სია)	კატალოგში დაბრუნება	click -> /

P6.1 - სია

- shop_orders-დან, ახალი ზემოთ. ბარათზე: ნომერი (#10X-00002), თარიღი (toLocaleDateString), ნივთების რაოდენობა, Total, სტატუსის ბეჯი. ცარიელი: „You have no orders yet.“ + „Browse products“.

P6.2 - სტატუსის სიმულაცია

- სტატუსი **არ ინახება** - ითვლება გასული დროიდან: elapsed = now - placedAt; < 1 წთ -> Processing; 1-3 წთ -> Shipped; > 3 წთ -> Delivered. ეს სუფთა ფუნქციაა getOrderStatus(order, now) ან Order კლასის მეთოდი (T4) - ერთი შეყვანა, ერთი პასუხი, ტესტირებადი.
- OrdersPage ყოველ 30 წამში გადაითვლის სტატუსებს (useEffect + setInterval(30 000), cleanup-ით unmount-ზე) - შეკვეთა შენს თვალწინ „იგზავნება“ გადატვირთვის გარეშე. დაცვაზე: „რა მოხდება cleanup-ის გარეშე?“

P6.3 - დეტალები (/orders/:id)

- useParams -> id -> find() shop_orders-ში; ვერ იპოვა -> NotFound. state.justPlaced -> მწვანე ბანერი (P5.3).
- ბლოკები: სათაური „Order #10X-00002“ + თარიღი; სტატუსის timeline (Processing ✓ -> Shipped ✓ -> Delivered ○); ნივთების ცხრილი (thumbnail, სათაური, რაოდენობა, ერთეულის ფასი, ხაზის ჯამი); მიწოდების მისამართი; გადახდის მეთოდი (Card •••• 4242 / Cash on delivery); ჯამები; ღილაკი „Reorder“.

Out of Scope

- შეკვეთის გაუქმება; ინვოისის PDF (ბონუსი); შეკვეთების ფილტრაცია / ძებნა; ისტორიის ჩატვირთვა /carts/user/{id}-დან (ბონუსი).

Measurable Outcome

✔ განთავსების შემდეგ შეკვეთა სიის თავში ჩანს სტატუსით Processing; ~1 წუთში Shipped ხდება გადატვირთვის გარეშე

✔ /orders/123456 (არარსებული) NotFound-ს აჩვენებს; Reorder ზუსტად იმავე ნივთებს ამატებს კალათაში

✔ ლოგაუთი -> ლოგინი: შეკვეთები ადგილზეა; Order ობიექტში ბარათის სრული ნომერი არ არის

P7 - Profile გვერდი (/profile) [FULL]

CORE — სავალდებულო მოთხოვნა არ არის.
FULL — Profile მონაცემები, რედაქტირება, Theme, Clear local data და დამატებითი მოქმედებები.

ღილაკების რეესტრი

ელემენტი	რას ემსახურება	რა ხდება დაჭერისას / გამოყენებისას
A. Save changes	სახელის / კონტაქტის განახლება	[FULL] Profile-ის მონაცემები რედაქტირდება ადგილობრივ controlled form-ში; ცვლილება აისახება React state-ში. მომხმარებლის API-სთვის ჩაწერა სავალდებულო არ არის.
B. თემის გადამრთველი	Dark / Light	იგივე ThemeContext, რაც Header-ში - მეორე იმპლემენტაცია არ არსებობს
C. Clear local data	ლოკალური მონაცემების განულება	[FULL] მარტივი React state/controlled form; გარე ავტორიზაცია არ არის საჭირო.
D. Log out	სესიის დახურვა	იგივე, რაც Header-ში (P0.3)

FULL: არსებული AuthContext-ის მარტივი username შეიძლება გამოჩნდეს პროფილში; რთული მომხმარებლის API ინტეგრაცია არ მოითხოვება.

- FULL: Profile აჩვენებს username-ს, სურვილისამებრ შეკვეთების რაოდენობას; API-დან მომხმარებლის აღდგენა არ მოითხოვება.

P7.2 — პროფილის რედაქტირება (controlled form, FULL)

ველი	ვალიდაციის წესი	შეცდომის ტექსტი (ზუსტი)
First name	სავალდებულო; trim()-ის შემდეგ მინიმუმ 2 სიმბოლო	First name must be at least 2 characters
Last name	სავალდებულო; მინიმუმ 2 სიმბოლო	Last name must be at least 2 characters
Email	სავალდებულო; ვალიდური ფორმატი	Please enter a valid email address
Phone	არასავალდებულო; თუ შევსებულია - 9-15 ციფრი	Please enter a valid phone number

- [FULL] Profile-ის მონაცემები რედაქტირდება ადგილობრივ controlled form-ში; ცვლილება აისახება React state-ში. მომხმარებლის API-სთვის ჩაწერა სავალდებულო არ არის.

Out of Scope

- პაროლის შეცვლა (DummyJSON-ზე აზრი არ აქვს); ავატარის ატვირთვა; ანგარიშის წაშლა.

Measurable Outcome

✔ FULL: პროფილზე ცვლილება React state-ში და Header-ში ჩანს.

✔ Clear local data ასუფთავებს კალათას, შეკვეთებსა და შეფასებებს, მაგრამ სესიას და თემას ინარჩუნებს

✔ FULL: პროფილის შეცვლისთვის გარე API ინტეგრაცია სავალდებულო არ არის.

A3. მოდულური მოთხოვნები: TypeScript (T), Tailwind CSS (TW), ოპტიმიზაცია (O), პროცესი (D)

T - TypeScript [მოდული: დინამიური ვებგვერდის შექმნა TypeScript-ის საშუალებით]

CORE — strict TypeScript; Product/Auth/Cart/Checkout მონაცემების, props-ის, state-ისა და ფუნქციების ტიპიზაცია; typecheck შეცდომების გარეშე.

FULL — Generics, custom ApiClient, ApiError, OOP კლასები, type guards, unions, utility types.

T1 - კონფიგურაცია [CORE]

- Vite-ის react-ts template; tsconfig.json-ში: "strict": true (რაც მოიცავს noImplicitAny-სა და strictNullChecks-ს), "noUnusedLocals": true, "noUnusedParameters": true, "noFallthroughCasesInSwitch": true, "jsx": "react-jsx". Path alias "@/*" -> src/* - ბონუსი (vite.config-შიც უნდა დაემატოს).
- package.json: "typecheck": "tsc --noEmit" და "lint": "eslint ." - ESLint typescript-eslint-ით და eslint-plugin-react-hooks-ით (rules-of-hooks: error, exhaustive-deps: warn). წესი @typescript-eslint/no-explicit-any: error - any კოდში ფიზიკურად ვერ დარჩება.
- unknown ნებადართულია იქ, სადაც ტიპი ნამდვილად უცნობია (catch (e: unknown), JSON.parse-ის შედეგი) - მაგრამ მხოლოდ type guard-ით გავიწროებული.

```
// tsconfig.json (ფრაგმენტი)
{
 "compilerOptions": {
  "target": "ES2020", "module": "ESNext", "jsx": "react-jsx",
  "strict": true, "noUnusedLocals": true, "noUnusedParameters": true,
  "noFallthroughCasesInSwitch": true, "skipLibCheck": true,
  "baseUrl": ".", "paths": { "@/*": ["src/*"] }
 }
}
```

T2 - მონაცემების ტიპიზაცია [CORE]; რთული union/utility ტიპები [FULL]

- types/ ფოლდერში დევს ყველა დომენური მოდელი (A1.5) - სხვაგან interface Product მეორედ არ იწერება. API-ს ყველა პასუხი ტიპიზებულია: api.get<ProductsResponse>(...), api.get<Product>(...).
- ყველა კომპონენტს აქვს Props interface (ProductCardProps, PaginationProps...); ყველა ფუნქციას - ექსპლიციტური დაბრუნების ტიპი იქ, სადაც ის აშკარა არ არის (async function fetchProducts(params: CatalogParams): Promise<ProductsResponse>).
- CORE: გამოიყენება მიმდინარე Product/Cart/Login/Checkout საჭიროების შესაბამისი ტიპები. [FULL] Literal unions: OrderStatus და SortOption (მხოლოდ იმ ფუნქციებისთვის, რომლებიც რეალურად შესრულებულია).
- **[FULL]** discriminated union-ები: AsyncState<T> და CartAction - switch-ი never-ით ამოწმებს, რომ ყველა შემთხვევა დაფარულია (P4.2-ის მაგალითი). Utility types მინიმუმ ერთგან გააზრებულად: Pick / Omit / Partial (მაგ: type CartItem = Pick<Product, "id" | "title" | ...> & { quantity: number } ან type ProfileForm = Pick<AuthUser, "firstName" | "lastName" | "email" | "phone">).

- **interface vs type - შენი წესი:** ობიექტის ფორმა, რომელიც შეიძლება გაფართოვდეს ან კლასმა განახორციელოს -> interface; union, intersection, alias -> type. დაცვაზე ამას ზუსტად შენს კოდზე გკითხავენ.

T3 - ძირითადი ტიპიზებული ფუნქციები [CORE]; generics/type guards [FULL]

- utils/format.ts: formatPrice(amount: number, currency = "USD"): string (Intl.NumberFormat); discountedPrice(price: number, pct: number): number; formatDate(iso: string): string. utils/url.ts: buildProductsUrl(params: CatalogParams): string. ყველა - სუფთა, ტიპიზებული, ერთი პასუხისმგებლობით.
- **Generics:** ApiClient.get<T>(path): Promise<T>; useFetch<T>(url): AsyncState<T>; useLocalStorage<T>(key: string, initial: T): [T, (v: T) => void]; StorageService<T>. generic-ი აქ შინაარსიანია - ერთი კოდი, ბევრი ტიპი.
- **Type guards:** isApiError(e: unknown): e is ApiError; isProduct(x: unknown): x is Product (localStorage-დან წაკითხულის შემოწმება - JSON-ს ტიპი არ აქვს!). as cast-ი guard-ის ნაცვლად - ქულის კლება.
- Optional chaining (?.) და nullish coalescing (??) გამოიყენე ტიპების საჭიროებისამებრ; non-null assertion (!) მხოლოდ დასაბუთებულ შემთხვევაში.
- [FULL] Enum ან as const კონსტანტები რეალურად გამოყენებული storage გასაღებებისთვის, მაგალითად export const STORAGE_KEYS = { cart: "shop_cart", orders: "shop_orders" } as const.

T4 - OOP, ApiClient და ApiError [FULL]

React კომპონენტები ფუნქციებია. კლასები **ბიზნეს-ლოგიკისთვის** იყენე - იქ, სადაც მონაცემი და მასზე ოპერაციები ერთად ცხოვრობს. „კლასი კლასისთვის“ არ ფასდება; ყოველ კლასს მიზეზი უნდა ჰქონდეს:

კლასი	რას აკეთებს	OOP-ის რა ელემენტები ჩანს
ApiClient (api/ApiClient.ts)	სასწავლო Login იყენებს აღგილობრივ React Context state-ს (username, isLoggedIn); სერვერული ავტორიზაცია, ავტორიზაციის ჰედერები და სესიის API არ გამოიყენება.	private readonly baseUrl; private request<T>() - საერთო ლოგიკა; public მეთოდები; genericsსაერთო ლოგიკა; public მეთოდები; generics
ApiError (api/ApiError.ts)	HTTP შეცდომა status-ით და message-ით; 404 -> NotFound, 401 -> logout, სხვა -> toast	extends Error (მემკვიდრეობა); constructor(public readonly status: number, message: string); super(message)
Cart (models/Cart.ts)	კალათის ყველა გამოთვლა (P4.3) - UI-ს გარეშე, ტესტირებადი	implements ICart (ინტერფეისის იმპლემენტაცია); private items; get itemCount() (getter); static-ი ფასდაკლების წესებისთვის (static readonly FREE_SHIPPING_FROM = 100)

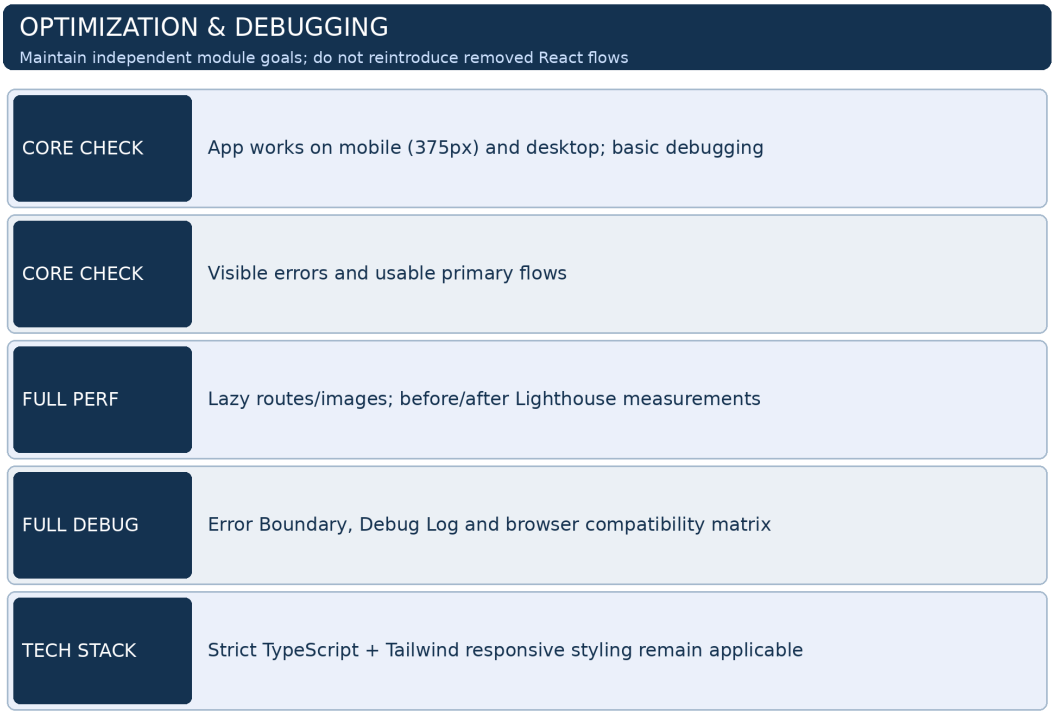
კლასი	რას აკეთებს	OOP-ის რა ელემენტები ჩანს
Order (models/Order.ts)	შეკვეთის შექმნა და სტატუსი: static create(cart: Cart, shipping: ShippingInfo, ...): Order; getStatus(now = new Date()): OrderStatus; toJSON() localStorage-სთვის	static factory; მეთოდი, რომელიც state-ს არ ცვლის; სერიალიზაცია
StorageService<T> (models/StorageService.ts)	localStorage-ის ერთადერთი კარიბჭე: get(): T | null (try/catch JSON.parse-ზე), set(value: T), remove(). ინსტანციები: cartStorage = new StorageService<CartState>("shop_cart") და ა.შ.	generic კლასი; private readonly key; ერთი პასუხისმგებლობა

```
// api/ApiClient.ts (ჩონჩხი - შეავსე შენ)
export class ApiClient {
 // საჯარო პროდუქტების API არ მოითხოვს მომხმარებლის ავტორიზაციას.
 constructor(private readonly baseUrl: string) {}


 private async request<T>(path: string, init: RequestInit = {}) {
  const headers: Record<string, string> = {
   "Content-Type": "application/json",
  };

  const res = await fetch(this.baseUrl + path, { ...init, headers });
  if (!res.ok) throw new ApiError(res.status, await res.text());
  return (await res.json()) as T;
 }
 get<T>(path: string, signal?: AbortSignal): Promise<T> {
  return this.request<T>(path, { signal });
 }
 post<T, B>(path: string, body: B): Promise<T> {
  return this.request<T>(path, {
   method: "POST", body: JSON.stringify(body),
  });
 }
}

// models/Cart.ts (ჩონჩხი)
export interface ICart {
 subtotal(): number; discount(): number; shipping(): number; total(): number;
}
export class Cart implements ICart {
 static readonly FREE_SHIPPING_FROM = 100;
 static readonly SHIPPING_COST = 10;
 constructor(
  private readonly items: CartItem[],
  private readonly promoCode: string | null,
 ) {}
 get itemCount(): number {
  return this.items.reduce((n, i) => n + i.quantity, 0);
 }
 subtotal(): number { /* Σ discountedPrice × quantity */ }
 // ...
}
```



Measurable Outcome

✔ npm run typecheck -> 0 შეცდომა; npm run lint -> any არ არის; შემფასებელი types/-ში ყველა მოდელს ერთ ადგილას პოულობს

✔ [FULL] ApiClient და ApiError რეალურად გამოიყენება პროდუქტების API-ში; 404 და ქსელური შეცდომები გასაგებად მუშავდება.

✔ [FULL] თუ Cart, Order და StorageService<T> კლასები რეალურად გამოიყენება, მათი interface/implements და მეთოდები მოწმდება; CORE-ში მათი შექმნა საჭირო არ არის.

✔ სტუდენტი საკუთარ კოდზე ხსნის interface vs type-ს, ერთ generic-ს და ერთ type guard-ს

O - ოპტიმიზაცია, Debugging და თავსებადობა [მოდული: ვებ გვერდის ოპტიმიზაცია]

CORE — მუშაობადი responsive გვერდები, ძირითადი debugging; შესავლის თანახმად Error Boundary და DEBUG-LOG-იც შედის მინიმალურ მოთხოვნებში.
FULL — lazy routes/images, Lighthouse before/after, მულტიბრაუზერული დოკუმენტაცია და რთული performance გაუმჯობესებები.

O1 - წარმადობა [FULL]; CORE-ში საბაზისო გამართულობა

- ① **Route-level code splitting:** გვერდები React.lazy(() => import("./pages/...")) -ით + ერთი <Suspense fallback={<Spinner />}> Layout-ში. შედეგი: პირველი ჩატვირთვა კატალოგის chunk-ს იღებს, /orders-ის კოდი - მხოლოდ მაშინ, როცა იქ გადახვალ (Network-ში ჩანს).
- ② **სურათები:** loading="lazy" ყველა thumbnail-ზე + width/height ატრიბუტები (CLS-ის წინააღმდეგ); ბადეში thumbnail, დეტალებში - სრული ფოტო; alt ყველგან.

- ③ **პაგინაცია და ზომიერი მონაცემები:** 194 პროდუქტი ერთად არასდროს იტვირთება; მხოლოდ საჭირო ველები ჩანს.
- ④ **debounce + AbortController** ძებნაზე (P2.2, P2.4) - ნაკლები რექვესთი, ნული „ძველი“ პასუხი.
- ⑤ **მემოიზაცია - მხოლოდ გაზომვით:** React.memo(ProductCard), useMemo ჯამებსა და წარმოებულ სიებზე (საშუალო რეიტინგი, გაფილტრული related), useCallback იმ handler-ებზე, რომლებიც memo-იზებულ შვილებს გადაეცემა. ყოველი memo-ს წინ React Profiler-ით ნახე, რომ ის ნამდვილად ამცირებს re-render-ებს - „memo ყველგან“ შეცდომაა, რომელზეც დაცვაზე გკითხავენ: „რატომ არის ეს useMemo აქ?“
- ⑥ **bundle:** vite build-ის output-ს დააკვირდი; ორიენტირი - მთავარი chunk ≤ 250 kB gzip. მძიმე ბიბლიოთეკა უმიზეზოდ - არა (moment.js - არა; date-fns ან Intl - კი).
- **Lighthouse (CORE - ანგარიში; FULL - სამიზნეები):** deploy-ზე, ინკოგნიტო ფანჯარაში, Mobile პრესეტით; სამიზნეები - Performance ≥ 85, Accessibility ≥ 90, Best Practices ≥ 90, CLS < 0.1. ანგარიშები (HTML ან სქრინები) რეპოში: docs/lighthouse/before.png და after.png + ცხრილი REPORT.md-ში: მეტრიკა | before | after | რა შეიცვალა.

O2 - Debugging [CORE: Error Boundary + DEBUG-LOG; FULL: გაფართოებული ანალიზი]

- **ინსტრუმენტები, რომლებიც უნდა გამოიყენო და აჩვენო:** Console (production კოდში console.log არ რჩება); Network (status, payload, timing, Offline რეჟიმი); Sources - breakpoint-ები (არა მხოლოდ console.log); React DevTools - Components (props / state / hooks ცოცხლად) და Profiler (რატომ რენდერდება); TypeScript-ის კომპილატორი - პირველი debugger-ი; ESLint-ის react-hooks/exhaustive-deps.
- **Error Boundary** (P0.6) - კლასური კომპონენტი static getDerivedStateFromError + componentDidCatch-ით; fallback „Something went wrong.“ + „Reload“. შემფასებელმა შეიძლება სთხოვოს კომპონენტში შეცდომის „ჩაგდება“ - fallback უნდა გამოჩნდეს.
- **DEBUG-LOG.md** - მინიმუმ 3 (CORE) / 5 (FULL) **ნამდვილი** ბაგი პროექტიდან, თითო ამ ფორმატით: **სიმპტომი** (რა ჩანდა) -> **რეპროდუქცია** (ნაბიჯები) -> **ინსტრუმენტი** (რომელი და რა აჩვენა) -> **მიზეზი** (root cause) -> **ფიქსი** (commit-ის ბმული) -> **რა ვისწავლე**.
- **[FULL] მრავალფეროვნება:** მინიმუმ ერთი Network-ის ბაგი (მაგ: 404 არასწორ endpoint-ზე), ერთი TypeScript-ის შეცდომა, რომელმაც runtime ბაგი აიცილა, ერთი re-render-ის პრობლემა Profiler-ით, ერთი routing / deploy-ის ბაგი (SPA rewrite, სესიის „გაფრენა“ გადატვირთვაზე).

O3 - Responsive [CORE]; cross-browser მატრიცა და a11y აუდიტი [FULL]

- **Responsive - 3 breakpoint:** mobile ≤ 480px (1 სვეტი, burger / დაკეცილი ნავიგაცია), tablet 481-1024px (2-3 სვეტი), desktop > 1024px (4 სვეტი). 375px სიგანეზე **ყველა გვერდი** გამოსადეგია, ჰორიზონტალური scroll არ არის, დასაჭერი ელემენტები ≥ 44px.
- **Cross-browser:** Chrome + Firefox (CORE), + Safari ან Edge (FULL). COMPATIBILITY.md - მატრიცა გვერდი × ბრაუზერი × მოწყობილობა (desktop / mobile; მობილური - DevTools-ის device emulation ან რეალური ტელეფონი) შედეგით (✓ / პრობლემა + ფიქსი) და სქრინებით docs/compat/-ში.

- **Accessibility-ის საფუძვლები:** სემანტიკური ტეგები (header / nav / main / footer); alt ყველა სურათზე; label ყველა ველზე (htmlFor + id); focus ხილულია; ფერების კონტრასტი (Lighthouse ამოწმებს). **[FULL]** კლავიატურით სრული გზა: Tab კატალოგზე -> Enter ხსნის პროდუქტს -> Tab-ით Add to cart -> Checkout-ის ფორმა კლავიატურით ივსება.

Measurable Outcome

✔ [FULL] Network-ში ჩანს route-level chunks; დამატებითი გვერდების chunks იტვირთება შესაბამისი მარშრუტის გახსნისას.

✔ Lighthouse-ის ანგარიშები რეპოშია; FULL-ზე deploy-ზე ცოცხლად გაშვებული Lighthouse სამიზნეებს აკმაყოფილებს

✔ 375px-ზე 7-ვე გვერდი მუშაობს ჰორიზონტალური scroll-ის გარეშე; COMPATIBILITY.md-ის მატრიცა სქრინებით შევსებულია

✔ DEBUG-LOG-ის ჩანაწერები ნამდვილია - commit-ები არსებობს; Error Boundary-ის fallback დემოზე ჩანს

D - დაგეგმვა, პროცესი და ანგარიშგება [მოდული: პრაქტიკული პროექტი]

CORE — GitHub repository, რეალური Git ისტორია, README.md (EN), მოკლე PLAN.md და REPORT.md, deploy.
FULL — Issues/Projects, milestone-ები, PR flow, 3 progress note, დეტალური სტატისტიკა და დემო ვიდეო.
ეს მოდული არ აფასებს კოდს - ის აფასებს, **როგორ** მიხვედი შედეგამდე: დაგეგმე თუ არა, შეასრულე თუ არა გეგმა და როგორ ჩააბარე ანგარიში. სამივე მტკიცებულება რეპოში უნდა იდოს.

D1 - მოკლე დაგეგმვა [CORE]; WBS/Issues/Milestones [FULL]

- **PLAN.md** (ინგლისურად ან ქართულად) იწერება **პირველ კვირაში, ფუნქციონალის კოდამდე** - შემფასებელი commit-ის თარიღს ამოწმებს. სექციები: მიზანი და scope (CORE / FULL თითო მოდულზე); WBS - epic-ები (Auth, Catalog, Product, Cart, Checkout, Orders, Profile, Quality, Docs) -> ამოცანები შეფასებით (საათი) და კვირით; რისკები (მინიმუმ 3) შემარბილებელი ზომით (მაგ: „DummyJSON მიუწვდომელია -> ლოკალური JSON fallback“); Definition of Done თითო ამოცანაზე.
- **GitHub Projects დაფა** (To do / In progress / Done) + მინიმუმ **20 Issue**, ლეიბლებით მოდულის მიხედვით (react / typescript / performance / docs). ამოცანა PLAN.md-დან; დახურვა - PR-ით ან commit-ით.

D2 - Git ისტორია და deploy [CORE]; PR/Projects/progress notes [FULL]

- **[CORE] Git-ის რეალური, შინაარსიანი commit-ების ისტორია და სამუშაო deploy; commit-ების ან სამუშაო დღეების ფიქსირებული მინიმუმი არ არის. [FULL] საორიენტაციო გაფართოებული პრაქტიკა: ≥ 40 შინაარსიანი commit მინიმუმ 15 სხვადასხვა დღეს; შესაძლო პრეფიქსები: feat:, fix:, refactor:, perf:, docs:, test:.**
- **[FULL] სამი ყოველკვირეული progress note: docs/progress/week-1.md, week-2.md და week-3.md — რა გაკეთდა, რა იგეგმება, ბლოკერები და დახარჯული საათები. CORE-ში ეს ფაილები სავალდებულო არ არის.**
- **[FULL] feature branch-ები და PR-ები:** მინიმუმ 8 merge-ებული PR, თითო Issue-ზე მიბმული, 3-ხაზიანი აღწერით (რა / რატომ / როგორ შევამოწმო). main-ზე პირდაპირი commit-ი - მხოლოდ setup-სა და docs-ზე.

- deploy ახლდება მინიმუმ კვირაში ერთხელ – ლაივ ბმული ყოველთვის მუშა მდგომარეობაშია.

D3 – README + მოკლე REPORT [CORE]; დეტალური ანგარიში/ვიდეო [FULL]

- **README.md (ინგლისურად):** About (2-3 წინადადება), Features (CORE / FULL-ის მიხედვით), Tech Stack, Architecture (ფოლდერების ხე + მონაცემთა ნაკადის 3-4 წინადადება), How to Run, Scripts, Live Demo (ბმული), Test Account (emilys / emilyspass), Credits (ვინ დაგეხმარა).
- **REPORT.md – საბოლოო ანგარიში:** დაგეგმილი vs მიწოდებული (ჩეკლისტი მოდულებისა და CORE / FULL-ის მიხედვით, რა არ გაკეთდა და რატომ); მეტრიკების ცხრილი (Lighthouse before / after, bundle-ის ზომა, commit-ების, PR-ებისა და დახურული Issue-ების რაოდენობა, საათები epic-ების მიხედვით); 3 ყველაზე რთული პრობლემა და გადაწყვეტა; „რას გავაკეთებდი სხვანაირად"; AI-ის გამოყენების შეჯამება (რა მიანდე, რა უარყავი და რატომ – 5-6 ხაზი).
- **[FULL]** დემო ვიდეო 2-3 წუთი (ეკრანის ჩანაწერი სრული ფლოუთი, ბმული README-ში) + 5-სლაიდიანი პრეზენტაცია ან README-ზე დაფუძნებული walkthrough დაცვისთვის.
- ai-log.md (JS პროექტის ფორმატით) სავალდებულო არ არის, მაგრამ რეკომენდებულია – დაცვისთვის მოსამზადებელი საუკეთესო მასალაა.

Measurable Outcome

✔ [CORE] PLAN.md ასახავს მოკლე გეგმას, ხოლო Git ისტორია – რეალურ სამუშაო პროცესს. [FULL] GitHub Issues/Projects, მინიმუმ 20 Issue და შესაბამისი milestone-ები.

✔ [CORE] რეპოზიტორიაში ჩანს შინაარსიანი commit-ები და მუშა deploy. [FULL] ≥ 40 commit მინიმუმ 15 სხვადასხვა დღეს და ზუსტად 3 progress note (სამი კვირისთვის).

✔ [CORE] REPORT.md მოკლედ აღწერს დაგეგმილსა და შესრულებულს. [FULL] წარმოდგენილი დამატებითი მეტრიკები (Lighthouse, commit-ები, PR-ები, Issues) რეალურ მონაცემებს ემთხვევა.

✔ FULL: ≥ 8 PR Issue-ებზე მიბმული; დემო ვიდეო იხსნება და სრულ ფლოუს აჩვენებს

A3-TW. Tailwind CSS

CORE: Tailwind CSS v4 ინტეგრაცია, utility-first სტილიზაცია, Flex/Grid, responsive მინიმუმ ერთი breakpoint-ით, hover/focus variant-ები.
FULL: @theme, dark mode, custom variants/utilities და გაფართოებული ვიზუალური სისტემა.

TW1 — ინსტალაცია და ინტეგრაცია [CORE]

Vite + React + TypeScript პროექტში გამოიყენება Tailwind CSS v4, @tailwindcss/vite plugin და ძირითად CSS ფაილში @import "tailwindcss". utility classes სწორად მუშაობს JSX-ში. პროექტის build წარმატებით სრულდება.
მტკიცებულება: ProductCard ან Header-ში გამოყენებული utility კლასი რეალურად ცვლის UI-ს; ნაჩვენებია Vite-ის კონფიგურაცია და import.

TW2 — Utility-first Styling [CORE]

სავალდებულოა ProductCard, Catalog, Product Details, Cart და Header-ის ძირითადი სტილიზაცია Tailwind-ით: padding, margin, gap, typography, colors, border, radius, widths და spacing. HTML სტრუქტურა რჩება სემანტიკური.
მტკიცებულება: სულ მცირე სამი განსხვავებული ძირითადი UI კომპონენტი გაფორმებულია Tailwind utility classes-ით; სტუდენტს შეუძლია ახსნას საკუთარი className-ები.

TW3 — Layout და responsive [CORE]

მინიმუმ ერთი რეალური responsive breakpoint (მაგ.: md:) უნდა ცვლიდეს პროდუქტების grid-ის სვეტების რაოდენობას ან სხვა მნიშვნელოვან განლაგებას. გამოიყენება flex ან grid და

mobile-first მიდგომა. 375px სიგანეზე CORE გვერდებს არ უნდა ჰქონდეთ მნიშვნელოვანი ჰორიზონტალური scroll.
მტკიცებულება: სტუდენტი ცვლის viewport-ს 375px-დან desktop-მდე და აჩვენებს განლაგების ცვლილებას.

TW4 — Interaction states / variants [CORE]

სულ მცირე ერთ ინტერაქტიულ კომპონენტზე გამოყენებულია hover:, focus: ან disabled: variant. React მდგომარეობის მიხედვით ვიზუალური ცვლილება გასაგებია.
მტკიცებულება: ღილაკზე ან input-ზე შესაბამისი მდგომარეობის რეალური დემონსტრირება.

TW5 — Customization [FULL]

მინიმუმ ერთი საკუთარი design token განისაზღვრება @theme-ში და გამოიყენება ინტერფეისში. სურვილისამებრ გამოიყენება @layer base, @layer components ან @utility, შესაბამისი ამოცანისთვის. Tailwind v4-ში არ გამოიყენება არასწორი დირექტივები @base ან @components.
მტკიცებულება: @theme-ის განსაზღვრება და მისი გამოყენება კონკრეტულ React კომპონენტში.

TW6 — გაფართოებული შესაძლებლობები [BONUS]

Dark mode, custom variants, custom utilities, მრავალჯერ გამოყენებადი დიზაინის კომპონენტები და რთული responsive მდგომარეობები. BONUS ვერ ანაცვლებს TW1–TW4 CORE მოთხოვნებს.

მოთხოვნა	დონე
Tailwind CSS v4-ის ინტეგრაცია	CORE
Utility classes — colors, spacing, typography	CORE
Flexbox და Grid	CORE
Responsive Design — mobile-first, breakpoints	CORE
Hover, Focus და Disabled states	CORE
Custom Theme (@theme)	FULL
Dark Mode, Custom Variants	BONUS